COMBINATIONAL LOGIC CIRCUITS

PART – 1

Submitted By

Kukkala Bhavya Sree

Under Guidance of

Ms. Harika

Table of Contents

1. Project Overview 3
2. Introduction to Combinational Circuits 4
3. Full Adder Using Half Adder (Structural Verilog) 5
4. 2x1 Multiplexer (MUX) 13
5. 4x1 Multiplexer 20
6. 8x1 Multiplexer Using 2x1 MUX 28
7. 4:2 Encoder 36
8. Hardware Verification on Basys 3 42
9. Complete Vivado Flow Summary 43

1. Project Overview

This document presents the complete step-by-step Xilinx Vivado design flow for implementing combinational logic circuits on a Digilent Basys 3 FPGA development board. The circuits covered include a Full Adder, Multiplexers (2x1, 4x1, 8x1), a 2:4 Decoder, a 4:2 Priority Encoder, and a Binary-to-Gray Code Converter — all described in Verilog HDL.

Stage	Tool / Action	Purpose
1	Create Vivado Project	Set up workspace, target device (Artix-7)
2	Write RTL Verilog	Behavioral/structural description of circuits
3	Write Testbench	Functional simulation stimulus
4	Run Simulation	Verify logic correctness in XSim
5	Synthesis	Map Verilog to FPGA primitives (LUTs)
6	Implementation	Place & Route on Artix-7 fabric
7	Generate Bitstream	Create programming file (.bit)
8	Program FPGA	Download bitstream to Basys 3
9	Hardware Verification	Test all input combinations on board

1.1 Target Hardware

Component	Specification
Board	Digilent Basys 3
FPGA	Xilinx Artix-7 XC7A35T-1CPG236C
Logic Cells	33,280
I/O Standard	LVCMOS33 (3.3 V)
Inputs Used	Slide switches (SW0, SW1, ...) → input signals
Outputs Used	LEDs (LD0–LD4) → circuit outputs
Clock	Not used (combinational design)
Tool	Xilinx Vivado Design Suite 2023.x

2. Introduction to Combinational Circuits

A **combinational logic circuit** is a digital circuit whose output at any instant depends only on the present combination of inputs — not on any past inputs. There is **no memory**, no clock, and no feedback path. In Verilog HDL, combinational logic is described using **assign** statements (dataflow) or **always @(*)** blocks (behavioral).

2.1 Key Characteristics

- Output = f(current inputs only) — no state or history involved.
- No clock signal required for purely combinational blocks.
- Implemented on FPGA using Look-Up Tables (LUTs) in the Artix-7 fabric.
- Propagation delay exists from input change to stable output.
- Examples: adders, multiplexers, decoders, encoders, code converters.

2.2 Combinational vs Sequential — Quick Comparison

Feature	Combinational	Sequential
Memory	No	Yes (flip-flops/registers)
Clock	Not required	Usually required
Output depends on	Current inputs only	Inputs + past state
Feedback	No	Yes
Verilog keyword	assign / always @(*)	always @(posedge clk)
Examples	Adder, MUX, Decoder	Counter, Register, FSM

2.3 Vivado Project Setup

For each experiment, a new Vivado RTL Project is created targeting the Basys 3 board:

STEP 1

Launch Vivado & New Project

Open Vivado → Click 'Create Project' → Enter project name → Select 'RTL Project' → Do not specify sources at this time → Click Next.

STEP 2

Select Target Device

Family: Artix-7 | Package: cpg236 | Speed Grade: -1 | Select: xc7a35tcpg236-1 (Basys 3) → Click Next → Finish.

3. Full Adder Using Half Adder (Structural Verilog)

A **Half Adder** adds two 1-bit inputs and produces a Sum and Carry. Two half adders and one OR gate combine to form a Full Adder. This uses **structural (hierarchical) Verilog** — instantiating lower-level modules to build a higher-level design, which mirrors real hardware design methodology.

3.1 Vivado Source Files

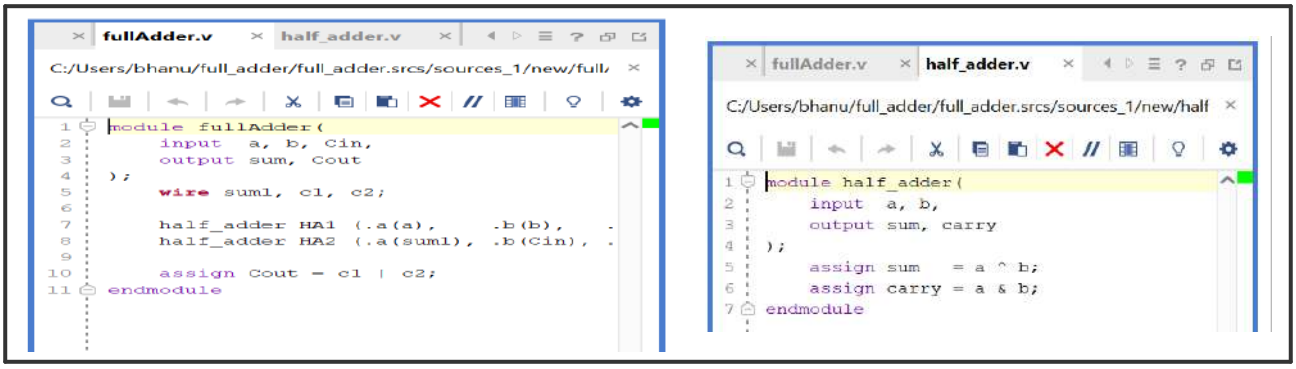


Figure 1 (Left): fullAdder.v — Structural RTL using HA1 & HA2 instances | Figure 2 (Right): half_adder.v Sub-module

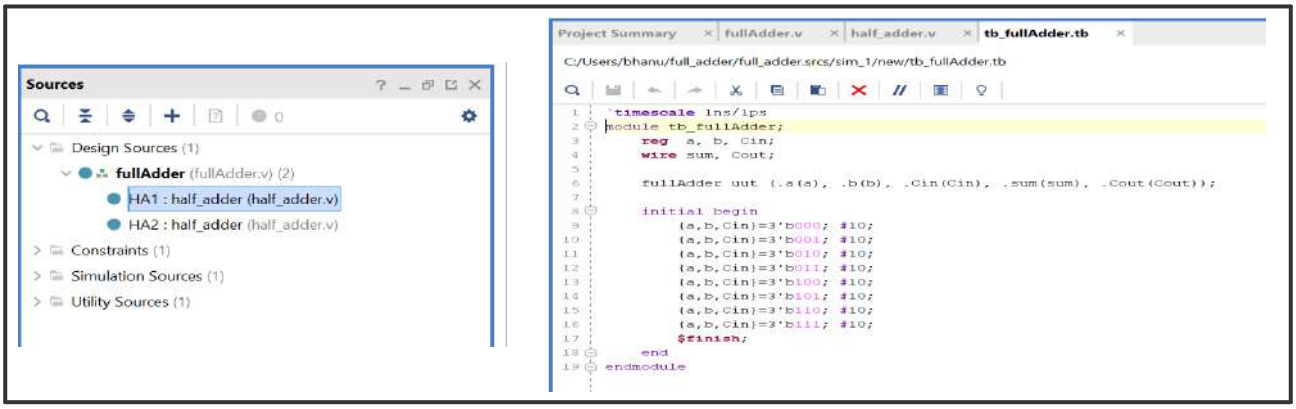


Figure 3 (Left): Sources Hierarchy — fullAdder (top) with HA1 & HA2 | Figure 4 (Right): tb_fullAdder.tb Testbench

3.2 Complete Verilog Source Code

half_adder.v — Sub-module

```
module half_adder(input a, b, output sum, carry);
    assign sum   = a ^ b;
    assign carry = a & b;
endmodule
```

fullAdder.v — Top Module (Structural)

```
module fullAdder(input a, b, Cin, output sum, Cout);
    wire sum1, c1, c2;
    half_adder HA1 (.a(a),    .b(b),    .sum(sum1), .carry(c1));
    half_adder HA2 (.a(sum1), .b(Cin), .sum(sum),  .carry(c2));
    assign Cout = c1 | c2;
endmodule
```

3.3 Block Structure

Block	Inputs	Outputs	Function
Half Adder 1 (HA1)	a, b	sum1, c1	First partial sum and carry
Half Adder 2 (HA2)	sum1, Cin	sum, c2	Add carry-in to partial sum
OR Gate	c1, c2	Cout	Generate final carry-out

3.4 XSim Simulation Waveform

Figure 5: XSim Waveform — All 8 input combinations verified.

The waveform confirms all 8 combinations of a, b, Cin produce correct sum and Cout. At t=30ns (a=0,b=1,Cin=1): sum=0, Cout=1. At t=70ns (a=1,b=1,Cin=1): sum=1, Cout=1 — matching the full adder truth table exactly.

Time (ns)	a	b	Cin	sum	Cout	Status
0–10	0	0	0	0	0	PASS
10–20	0	0	1	1	0	PASS
20–30	0	1	0	1	0	PASS
30–40	0	1	1	0	1	PASS
40–50	1	0	0	1	0	PASS
50–60	1	0	1	0	1	PASS
60–70	1	1	0	0	1	PASS
70–80	1	1	1	1	1	PASS

Table: All 8 Test Cases PASS

All 8 test cases PASS. Simulation confirms correct Full Adder logic.

3.5 RTL & Synthesized Schematics

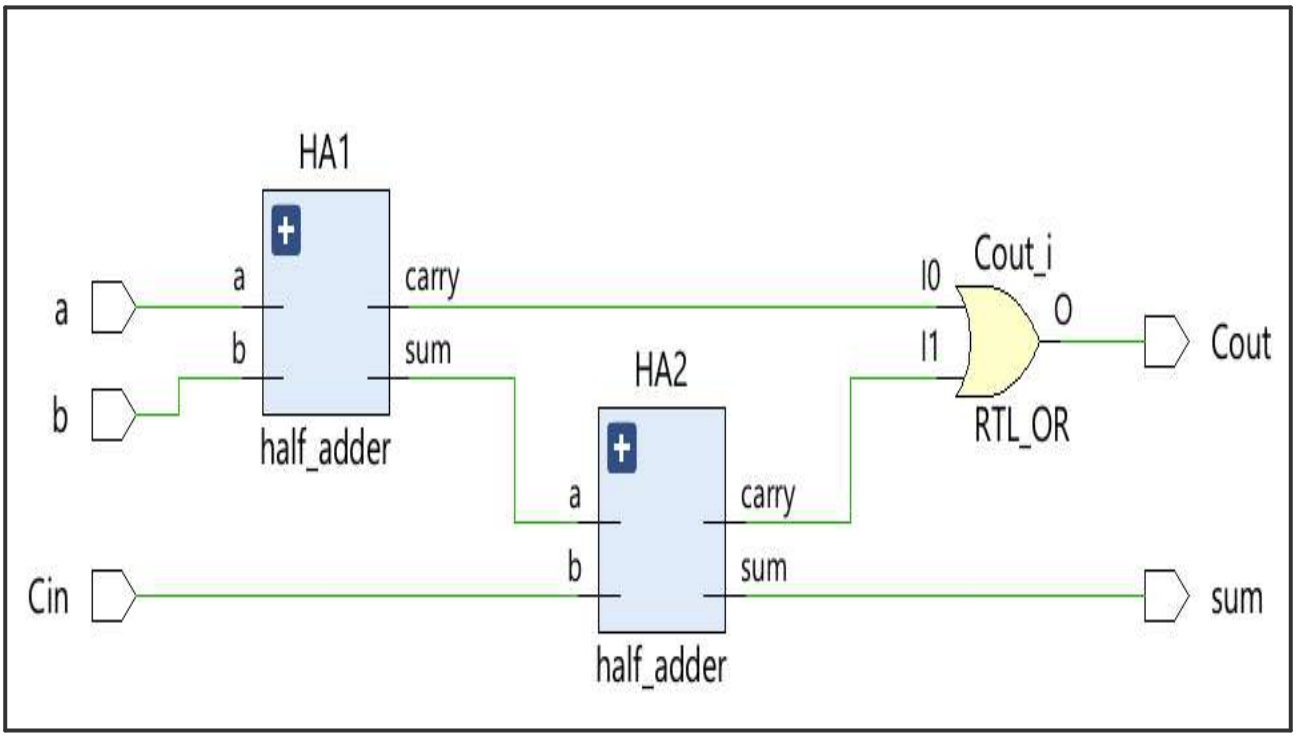


Figure 6: RTL Schematic — HA1, HA2 instances and RTL_OR gate

Vivado elaborates the structural code into HA1 (sum1 = a XOR b, c1 = a AND b), HA2 (sum = sum1 XOR Cin, c2 = sum1 AND Cin), and an OR gate combining c1 and c2 to produce Cout.

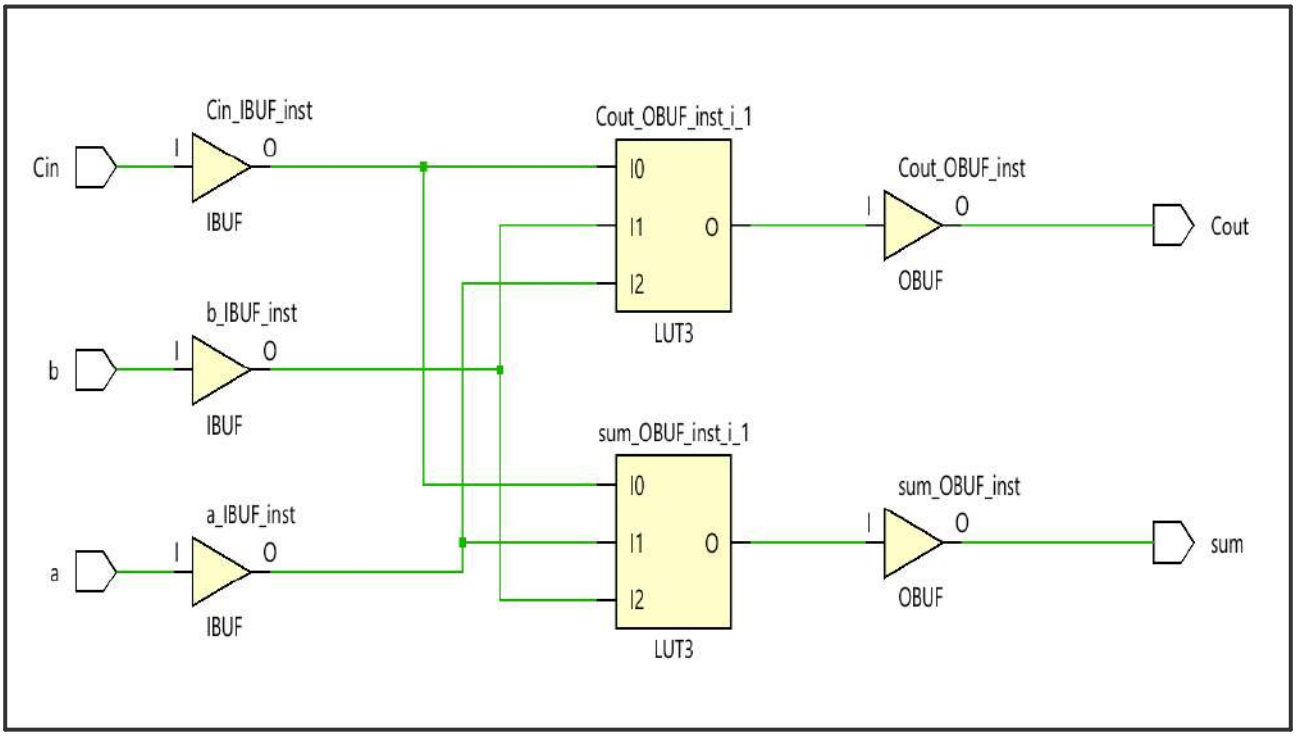
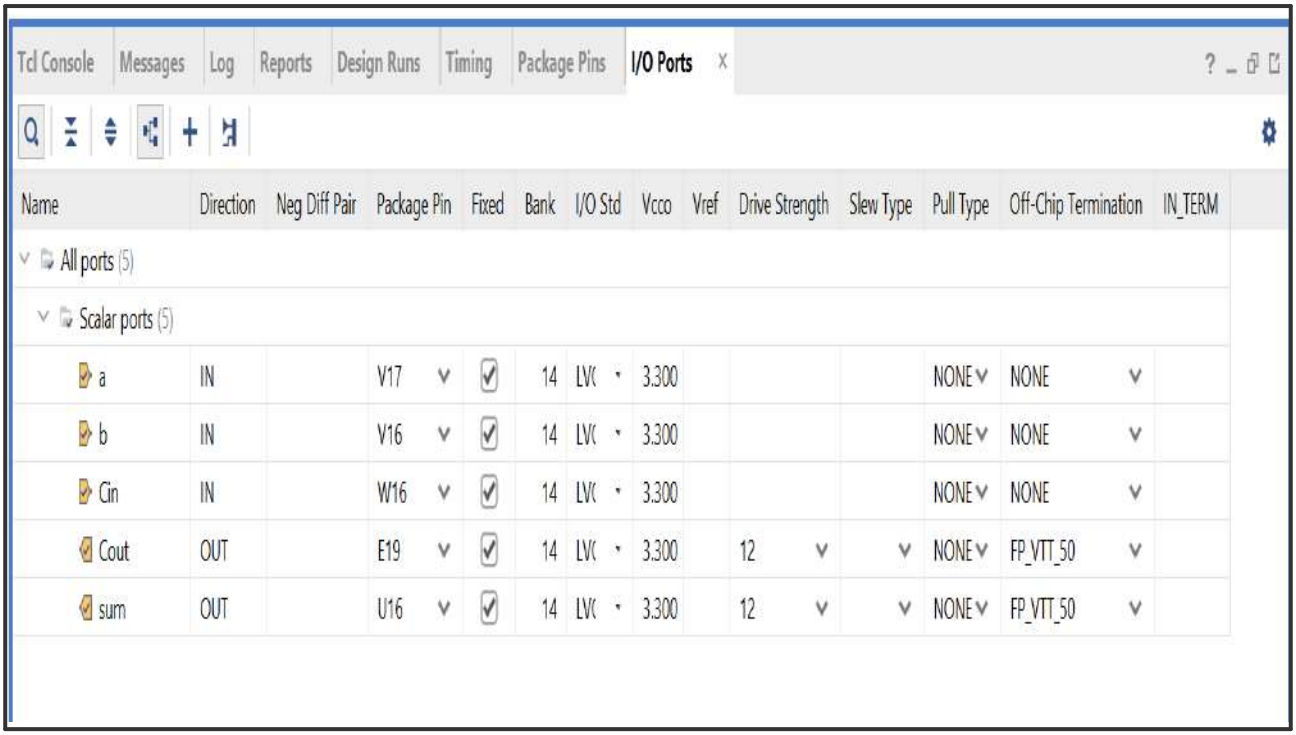


Figure 7: Synthesized Schematic — IBUF, LUT3 x2, OBUF

After synthesis, Vivado maps to two LUT3 primitives — one computing Cout and the other computing sum. IBUFs and OBUFs handle the I/O interface.

3.6 I/O Port Assignments



Tcl Console | Messages | Log | Reports | Design Runs | Timing | Package Pins | I/O Ports

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
All ports (5)													
Scalar ports (5)													
a	IN		V17	✓	14	LVC	3.300				NONE	NONE	
b	IN		V16	✓	14	LVC	3.300				NONE	NONE	
Cin	IN		W16	✓	14	LVC	3.300				NONE	NONE	
Cout	OUT		E19	✓	14	LVC	3.300		12		NONE	FP_VTT_50	
sum	OUT		U16	✓	14	LVC	3.300		12		NONE	FP_VTT_50	

Figure 8: I/O Ports — a=V17, b=V16, Cin=W16, Cout=E19, sum=U16

Inputs a, b, Cin map to SW0–SW2. Outputs sum and Cout drive LD0 (U16) and LD1 (E19). All ports use LVCMOS33 at 3.3V on Bank 14.

Port	Direction	Package Pin	Basys 3 Component	I/O Standard
a	Input	V17	Slide Switch SW0	LVCMOS33
b	Input	V16	Slide Switch SW1	LVCMOS33
Cin	Input	W16	Slide Switch SW2	LVCMOS33
sum	Output	U16	LED LD0	LVCMOS33
Cout	Output	E19	LED LD1	LVCMOS33

3.7 Implementation, Timing & Power

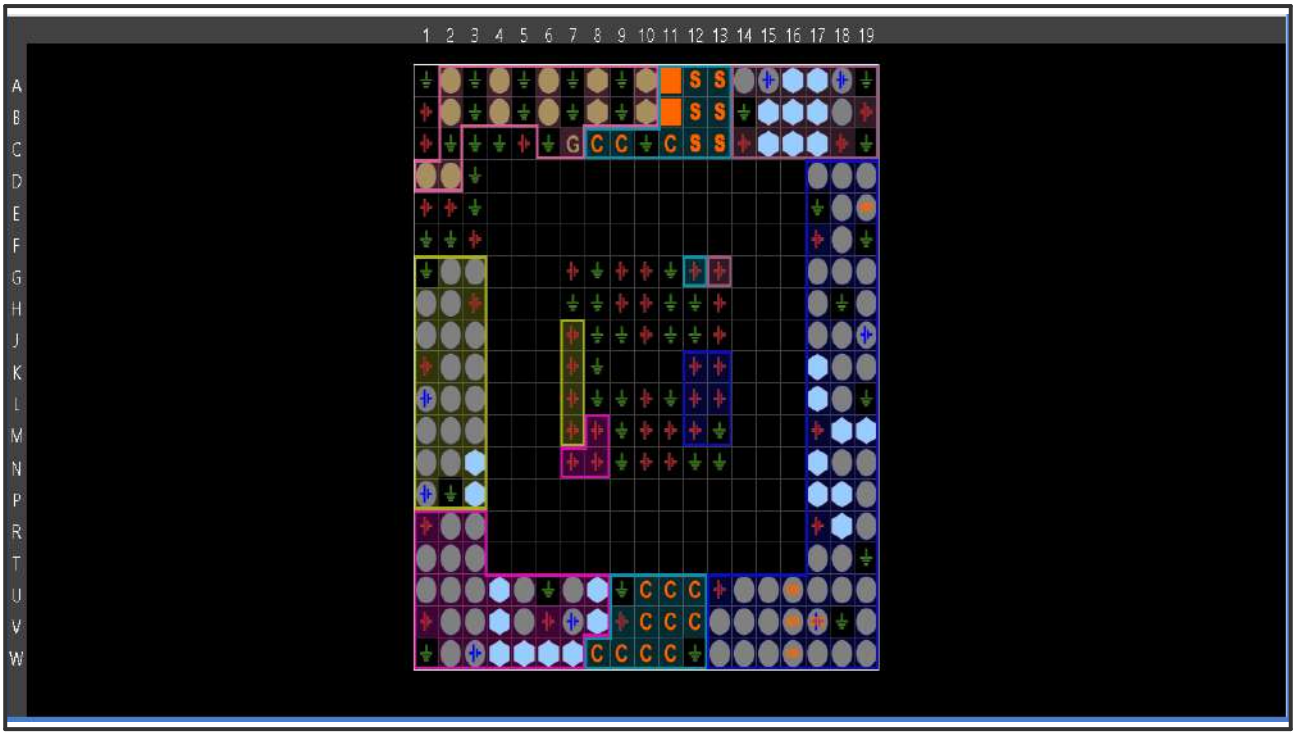


Figure 9: FPGA Package View — Place and Route on Artix-7

The package view shows I/O pads placed at edges for switch/LED pins. LUT cells are placed in the FPGA fabric interior. Routing is minimal, reflecting the compact 2-LUT implementation.

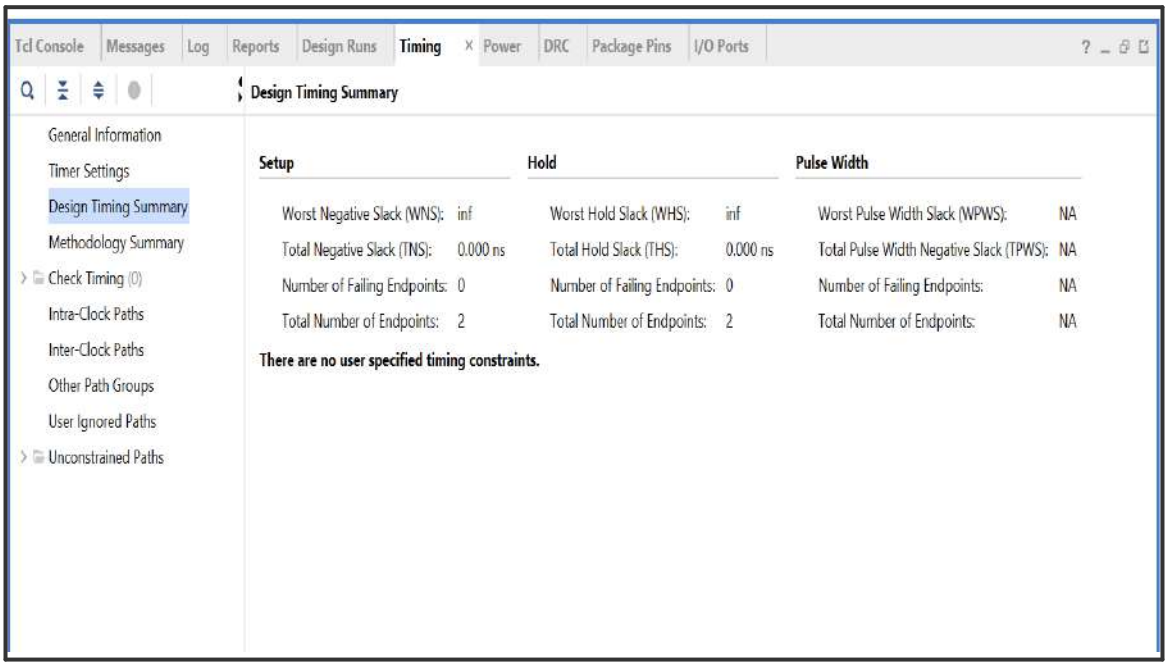


Figure 10: Timing Summary — WNS=inf, 0 failing endpoints

Pure combinational design with no clock. WNS=inf and TNS=0ns confirm no timing violations.

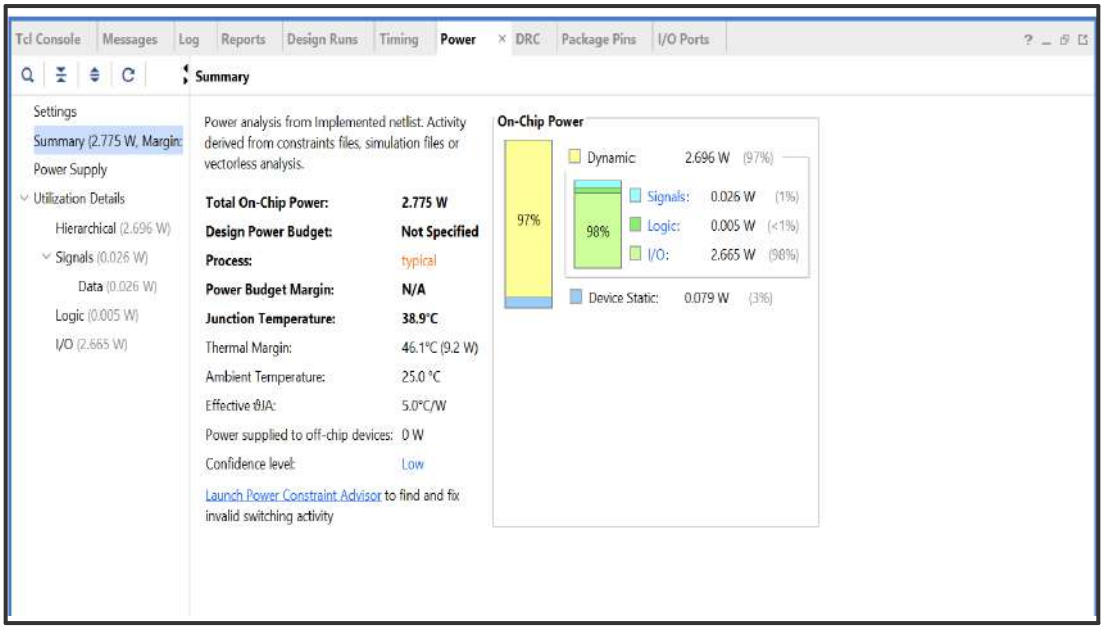


Figure 11: Power Analysis — Total 2.775 W

Total on-chip power 2.775 W; I/O dominates at 2.665 W (98% of dynamic). Logic uses only 0.005 W. Junction temperature 38.9°C with 46.1°C thermal margin.

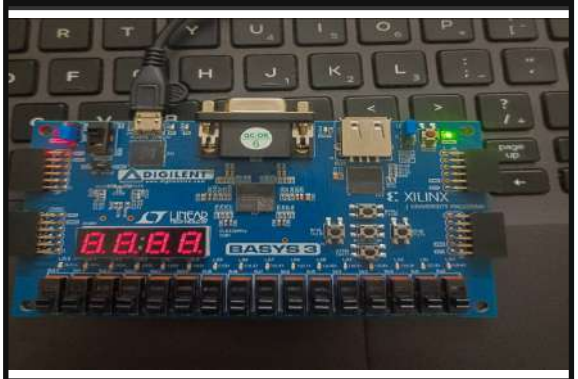
Power Component	Value	Percentage
Total On-Chip Power	2.775 W	100%
Dynamic Power	2.696 W	97%
— I/O	2.665 W	98%
Device Static	0.079 W	3%
Junction Temperature	38.9°C	—

Table: Full Adder (Structural) Power Report Summary

3.8 FPGA Output — Full Adder on Basys 3

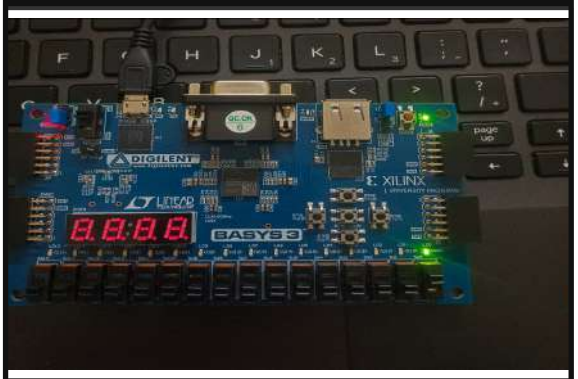
All 8 input combinations tested on Basys 3. **SW0=a (V17), SW1=b (V16), SW2=Cin (W16) → LD0=sum (U16), LD1=Cout (E19).**

TC1: a=0,b=0,Cin=0



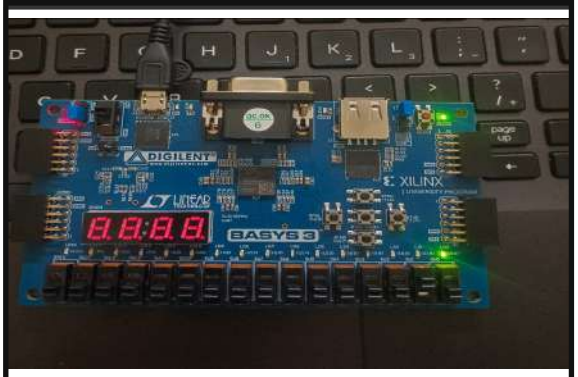
sum=0, Cout=0 | LEDs OFF

TC2: a=0,b=0,Cin=1



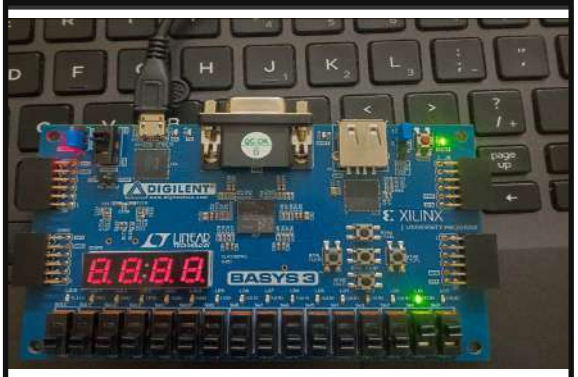
sum=1, Cout=0 | LD0 ON

TC3: a=0,b=1,Cin=0



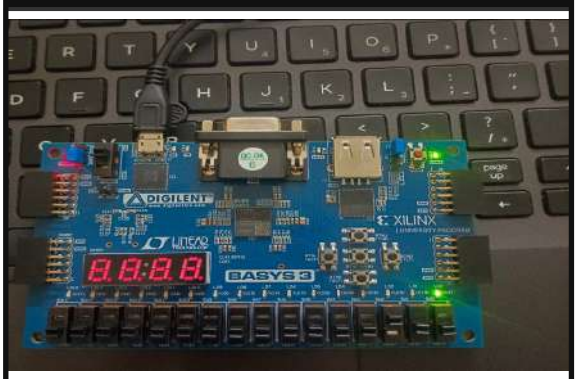
sum=1, Cout=0 | LD0 ON

TC4: a=0,b=1,Cin=1



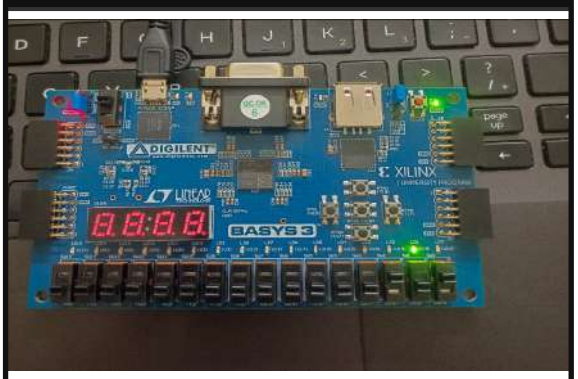
sum=0, Cout=1 | LD1 ON

TC5: a=1,b=0,Cin=0



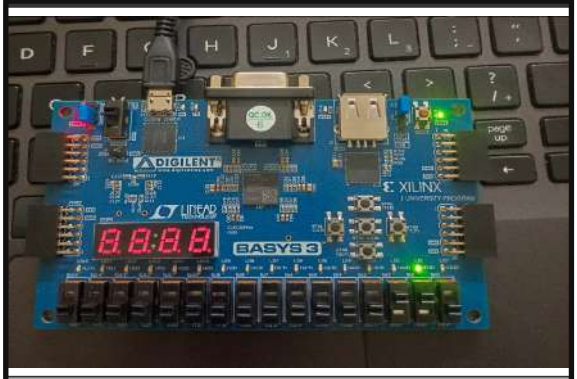
sum=1, Cout=0 | LD0 ON

TC6: a=1,b=0,Cin=1



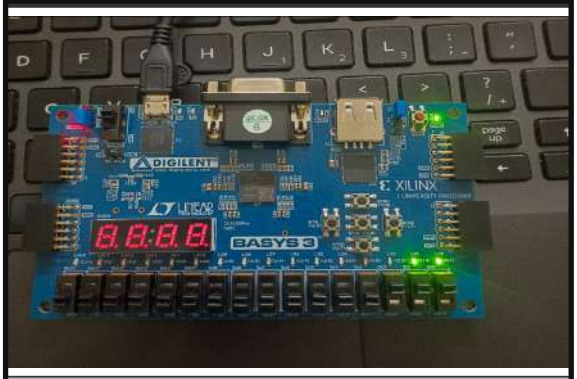
sum=0, Cout=1 | LD1 ON

TC7: a=1,b=1,Cin=0



sum=0, Cout=1 | LD1 ON

TC8: a=1,b=1,Cin=1



sum=1, Cout=1 | LD0+LD1 ON

Test Case	a	b	Cin	sum	Cout	Result
TC1	0	0	0	0	0	✓ PASS
TC2	0	0	1	1	0	✓ PASS
TC3	0	1	0	1	0	✓ PASS
TC4	0	1	1	0	1	✓ PASS
TC5	1	0	0	1	0	✓ PASS
TC6	1	0	1	0	1	✓ PASS
TC7	1	1	0	0	1	✓ PASS
TC8	1	1	1	1	1	✓ PASS

Table: Full Adder FPGA Hardware — All 8 Cases Verified

All 8 hardware test cases PASS. Full Adder using Half Adder verified on FPGA.

4. 2x1 Multiplexer (MUX)

A **Multiplexer (MUX)** is a data selector — it routes one of several input signals to a single output based on select lines. A **2x1 MUX** has 2 data inputs (I0, I1), 1 select line (s), and 1 output (y). When s=0, y=I0; when s=1, y=I1.

Boolean expression: **y = s'·I0 + s·I1**

5.1 Function Table

s (Select)	y (Output)
0	I0 (Input 0 selected)
1	I1 (Input 1 selected)

Table: 2x1 MUX Function Table

5.2 RTL Verilog Code

STEP 6

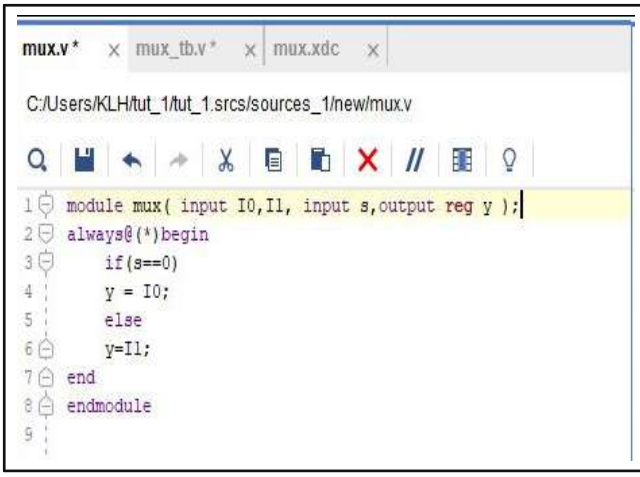
Create Verilog Source

Sources panel → '+' → Create Design Sources → Name: mux → OK → Finish

STEP 7

Write the RTL Code

Double-click mux.v in Sources panel and enter the code below:



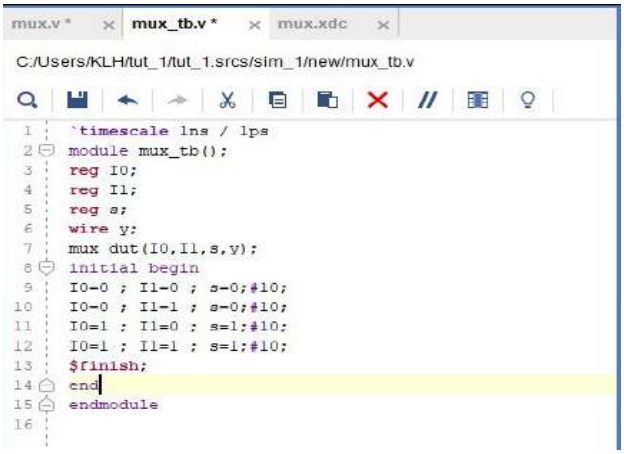


Figure 1: mux.v RTL Source Code in Vivado Editor (Left) | Figure 2: mux_tb.v Testbench (Right)

5.3 Complete Verilog Source Code

```
module mux(
    input  I0, I1,
    input  s,
    output reg y
);
    always @(*) begin
        if (s == 0)
            y = I0;
        else
            y = I1;
```

```
    end
endmodule
```

5.4 Design Explanation

Line	Statement	Logic Description
if (s==0)	Select condition	When select line is LOW, route I0 to output
y = I0	Output assignment	Input I0 is forwarded to y
else y = I1	Default condition	When select is HIGH, route I1 to output

5.5 Testbench Code

```
`timescale 1ns / 1ps
module mux_tb();
    reg I0, I1, s;
    wire y;
    mux dut(I0, I1, s, y);
    initial begin
        I0=0; I1=0; s=0; #10;   // s=0 → y=I0=0
        I0=0; I1=1; s=0; #10;   // s=0 → y=I0=0
        I0=1; I1=0; s=1; #10;   // s=1 → y=I1=0
        I0=1; I1=1; s=1; #10;   // s=1 → y=I1=1
        $finish;
    end
endmodule
```

5.6 Run Functional Simulation

STEP 8

Set Testbench as Top

Sources panel → Simulation Sources → right-click mux_tb → 'Set as Top'

STEP 9

Run Behavioral Simulation

Flow Navigator → Simulation → 'Run Behavioral Simulation'. Vivado launches XSim and opens the waveform viewer.

STEP 10

Add Signals & Run

In Scope panel select mux_tb → Objects panel, select all signals → right-click → 'Add to Wave Window' → Click Run ■ → Ctrl+Shift+F to Zoom Fit

5.7 Synthesis — RTL Schematic

STEP 11

Run Synthesis

Flow Navigator → Synthesis → 'Run Synthesis'. After completion → 'Open Synthesized Design'.

STEP 12

View RTL Schematic

Tools → Schematic or Flow Navigator → RTL Analysis → 'Open Elaborated Design' → Schematic

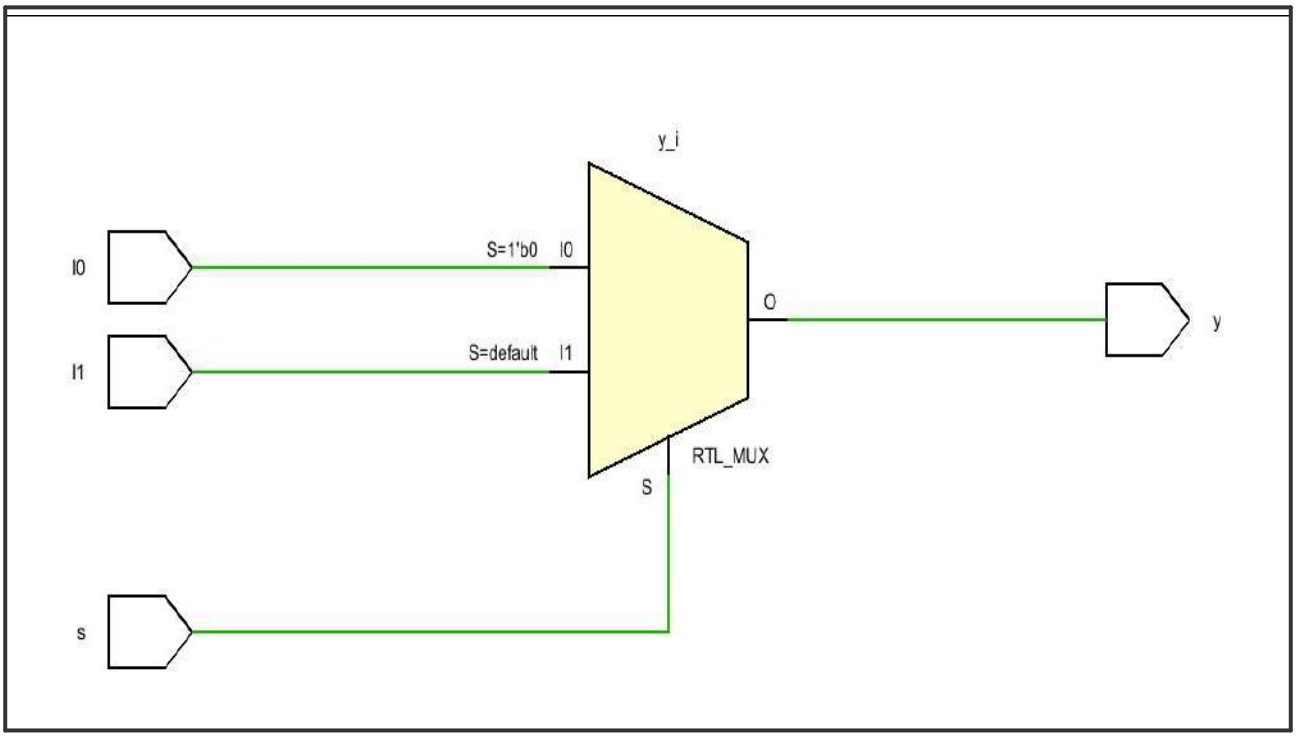


Figure 3: RTL Schematic — RTL_MUX with I0, I1, S, Y

Vivado elaborates the behavioral always block into an RTL_MUX primitive with three inputs (I0, I1, S) and one output (Y). The select line S controls which data input is routed to Y: when S=0, Y=I0; when S=1, Y=I1.

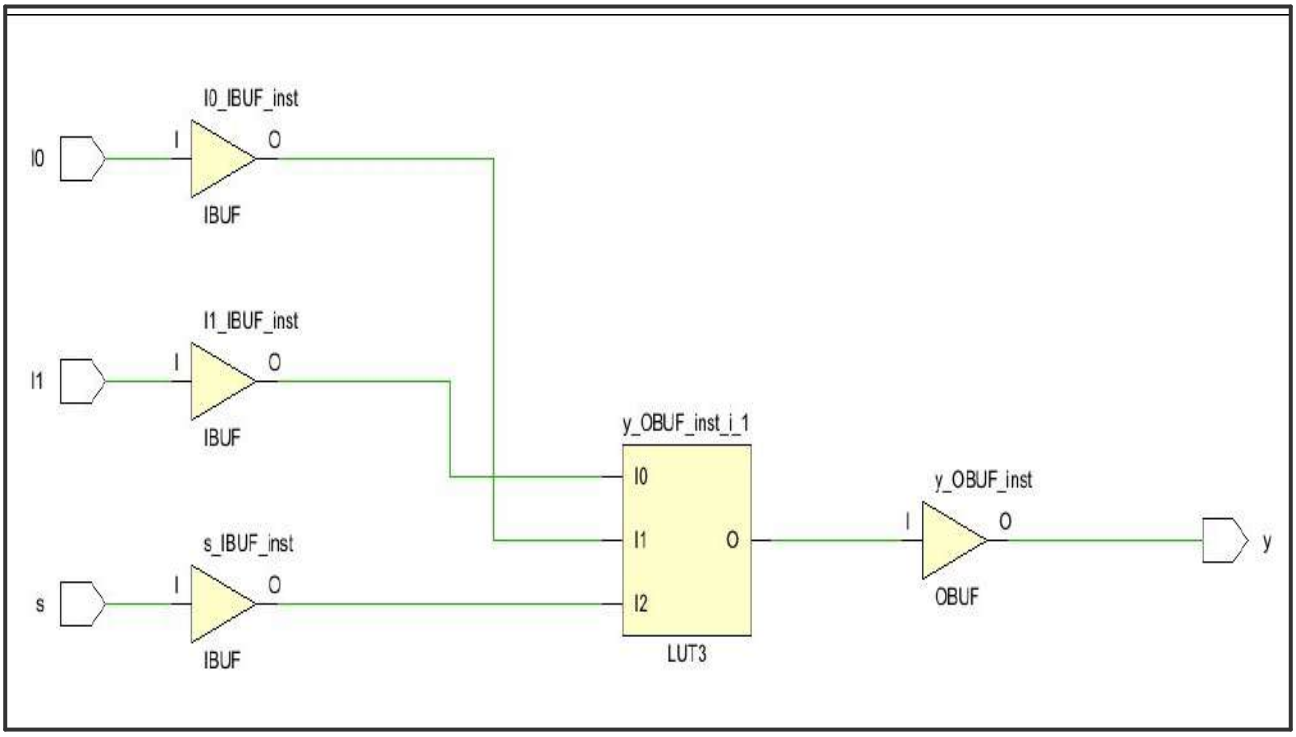


Figure 4: Synthesized Schematic — IBUF, LUT3, OBUF primitives

After synthesis the RTL_MUX maps to a single LUT3 on the Artix-7 fabric. The three IBUFs buffer I0, I1, S from the I/O pads into the FPGA core. The OBUF drives the output Y to the LED. Only 1 LUT3 out of 20,800 available is used — resource utilization < 0.1%.

5.8 Synthesis Results

Primitive	Count	Maps To	Notes
IBUF	3	I0, I1, s	Auto-inserted I/O input buffers
LUT3	1	y output	3-input Look-Up Table for MUX logic
OBUF	1	y	Auto-inserted output drive buffer

5.9 Add Pin Constraints (XDC)

STEP 13

Create Constraints File

Sources panel → '+' → Add or Create Constraints → 'Create File' → Name: constraints → OK → Finish

STEP 14

Enter Pin Constraints

Double-click constraints.xdc and enter the following:

```
## Inputs — Slide Switches
set_property PACKAGE_PIN V16 [get_ports {I0}]
    set_property IOSTANDARD LVCMOS33 [get_ports {I0}]
set_property PACKAGE_PIN V17 [get_ports {I1}]
    set_property IOSTANDARD LVCMOS33 [get_ports {I1}]
set_property PACKAGE_PIN W16 [get_ports {s}]
    set_property IOSTANDARD LVCMOS33 [get_ports {s}]
## Output — LED
set_property PACKAGE_PIN U16 [get_ports {y}]
    set_property IOSTANDARD LVCMOS33 [get_ports {y}]
```

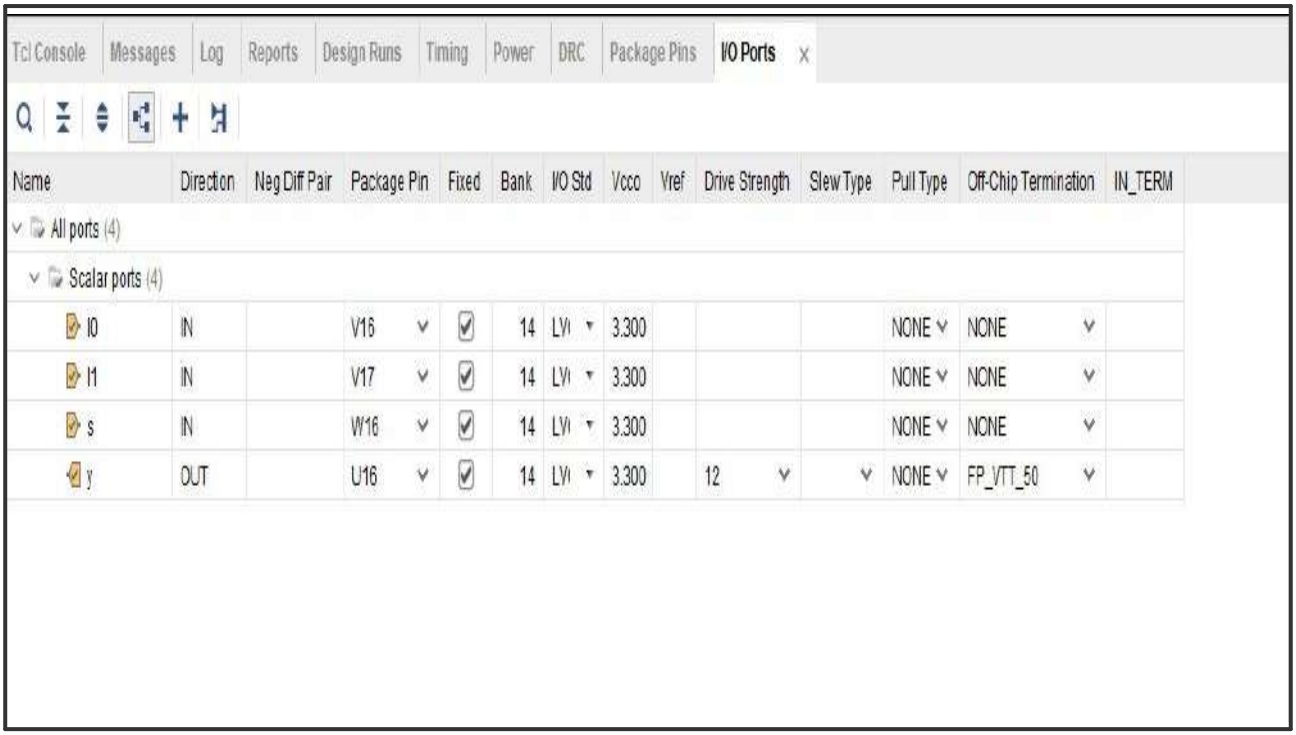


Figure 5: I/O Ports — V16=I0, V17=I1, W16=s, U16=y

The XDC constraints assign I0 to SW0 (V16), I1 to SW1 (V17), select s to SW2 (W16), and output y to LED LD0 (U16). All ports use LVCMOS33 I/O standard at 3.3V, compatible with the Basys 3 on-board switches and LEDs.

Port	Direction	Physical Pin	Basys 3 Component	I/O Standard
I0	Input	V16	Slide Switch SW0	LVCMOS33
I1	Input	V17	Slide Switch SW1	LVCMOS33
s	Input	W16	Slide Switch SW2	LVCMOS33
y	Output	U16	LED LD0	LVCMOS33

5.10 Implementation (Place & Route)

STEP 15

Run Implementation

Flow Navigator → Implementation → 'Run Implementation'. Click OK on dialogs. Takes 1–3 minutes.

STEP 16

Open Implemented Design

After completion → click 'Open Implemented Design'. Review Device view to see placed logic and routed nets.

5.11 Power Analysis

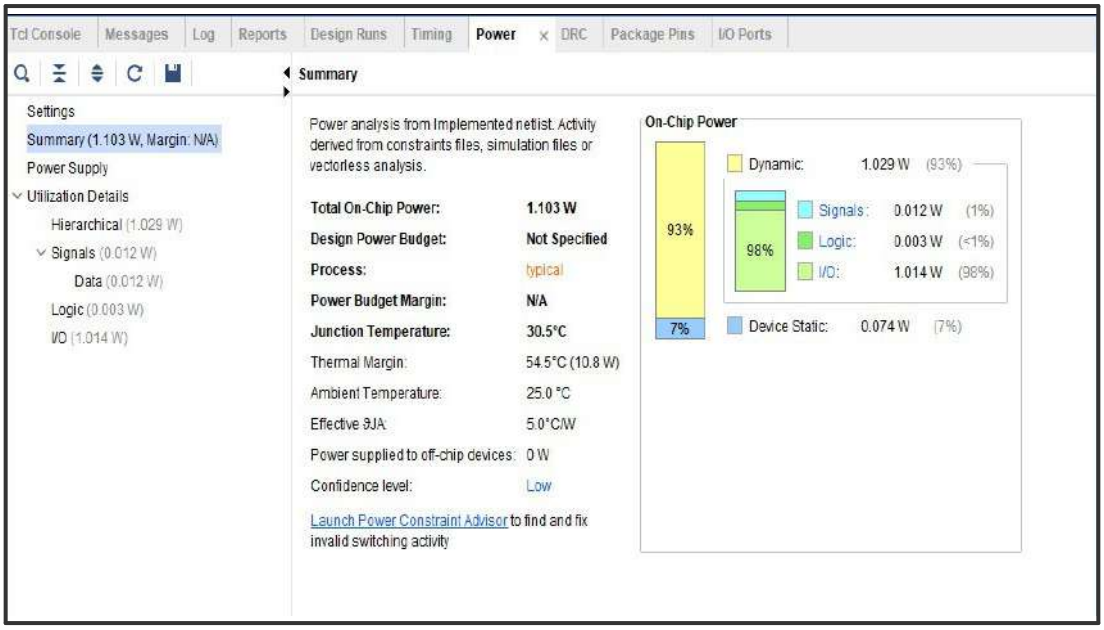


Figure 6: Power Analysis — Total 1.103 W

Total on-chip power is 1.103 W. I/O consumes 1.014 W (98% of dynamic power) due to the actively driven switch and LED pins. Logic power is only 0.003 W, confirming minimal FPGA fabric usage. Junction temperature is 30.5°C, well within the safe operating range.

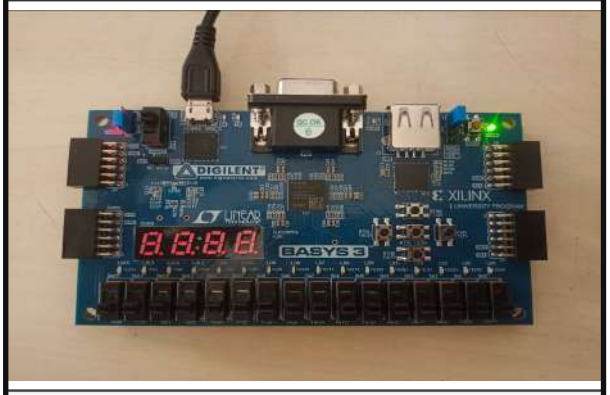
Power Component	Value	Percentage
Total On-Chip Power	1.103 W	100%
Dynamic Power	1.029 W	93%
— Signals	0.012 W	1%
— Logic (LUTs)	0.003 W	<1%
— I/O	1.014 W	98%
Device Static	0.074 W	7%
Junction Temperature	30.5°C	—
Ambient Temperature	25.0°C	—

Report	Key Finding
Timing	No timing constraints needed — combinational, no clock
Utilization	1 LUT3 used — <1% of 20,800 available on XC7A35T
DRC	No critical warnings — design is clean
Power	Total: 1.103 W | Dynamic: 1.029 W | Static: 0.074 W
Temperature	Junction: 30.5°C | Thermal margin: 54.5°C

5.12 FPGA Output — 2x1 MUX on Basys 3

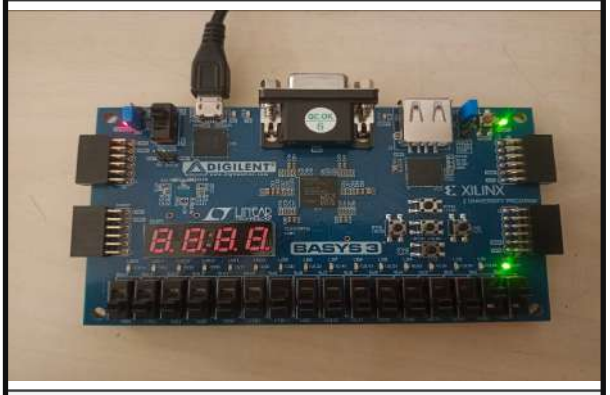
After programming the bitstream, all 4 input combinations were tested on the Basys 3 board. **SW0 (V16) = I0, SW1 (V17) = I1, SW2 (W16) = s (select), LD0 (U16) = y (output).**

Test Case 1: s=0, I0=0, I1=0



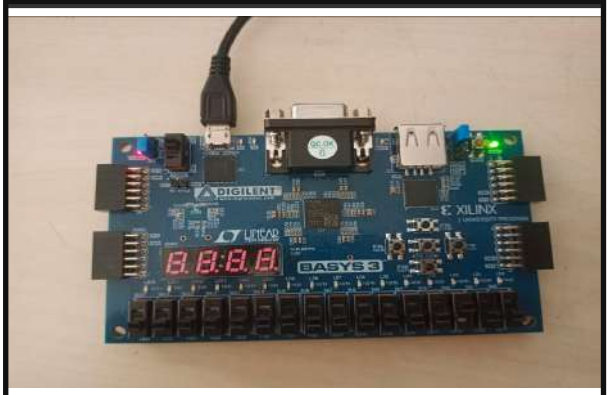
y = I0 = 0 | LD0 = OFF

Test Case 2: s=0, I0=0, I1=1



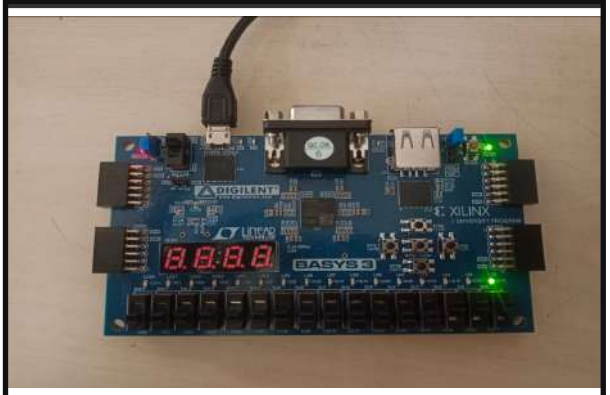
y = I0 = 0 | LD0 = OFF

Test Case 3: s=1, I0=1, I1=0



y = I1 = 0 | LD0 = OFF

Test Case 4: s=1, I0=1, I1=1



y = I1 = 1 | LD0 = ON

Test Case	s	I0	I1	y (Expected)	LD0 (Observed)	Result
Case 1	0	0	0	0	OFF	✓ PASS
Case 2	0	0	1	0	OFF	✓ PASS
Case 3	1	1	0	0	OFF	✓ PASS
Case 4	1	1	1	1	ON	✓ PASS

Table: 2x1 MUX FPGA Hardware Output — All 4 Test Cases Verified on Basys 3

All 4 hardware test cases PASS. The 2x1 MUX is verified correct on physical FPGA hardware.

5. 4x1 Multiplexer

A **4x1 MUX** selects one of four data inputs (i0–i3) based on a 2-bit select line (s1, s0) and routes it to the single output. This implementation uses **gate-level Verilog** — NOT, AND, and OR primitives — demonstrating structural design at the logic gate level.

s1	s0	Output
0	0	i0
0	1	i1
1	0	i2
1	1	i3

Table: 4x1 MUX Function Table

6.1 Vivado Source Files

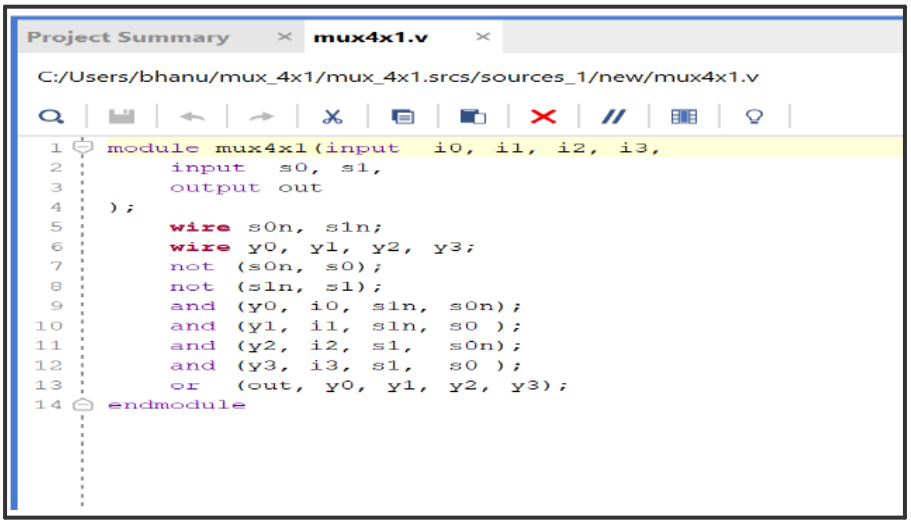


Figure 1: mux4x1.v — Gate-level implementation in Vivado

The design uses NOT gates for inverted selects (s0n, s1n), four AND3 gates for minterm generation (y0–y3), and a final 4-input OR gate. This is a gate-level structural approach: y = (i0·s1n·s0n) | (i1·s1n·s0) | (i2·s1·s0n) | (i3·s1·s0).

6.2 Complete Verilog Source Code

```
module mux4x1(input i0, i1, i2, i3,
              input s0, s1,
              output out
);
    wire s0n, s1n;
    wire y0, y1, y2, y3;
    not (s0n, s0);            // s0 inverted
    not (s1n, s1);            // s1 inverted
    and (y0, i0, s1n, s0n);  // out when s1=0,s0=0
    and (y1, i1, s1n, s0 );  // out when s1=0,s0=1
    and (y2, i2, s1,  s0n);  // out when s1=1,s0=0
    and (y3, i3, s1,  s0 );  // out when s1=1,s0=1
    or  (out, y0, y1, y2, y3);
```

```
endmodule
```

6.3 Testbench

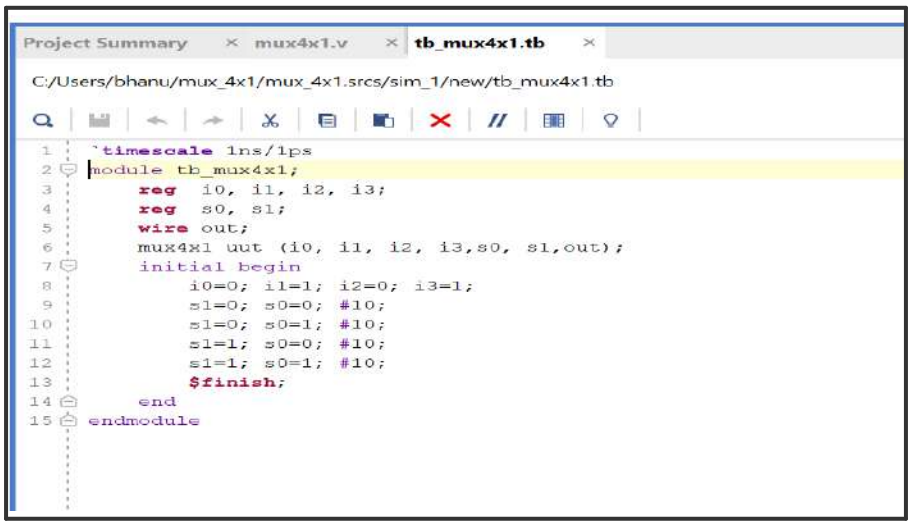


Figure 2: tb_mux4x1.tb — Testbench (i0=0,i1=1,i2=0,i3=1 fixed; s1,s0 varied)

The testbench fixes data inputs as i0=0, i1=1, i2=0, i3=1 and cycles through all four select combinations. Expected: s1=0,s0=0 → out=i0=0; s1=0,s0=1 → out=i1=1; s1=1,s0=0 → out=i2=0; s1=1,s0=1 → out=i3=1.

```
`timescale 1ns/1ps
module tb_mux4x1;
    reg i0, i1, i2, i3;
    reg s0, s1;
    wire out;
    mux4x1 uut (i0, i1, i2, i3, s0, s1, out);
    initial begin
        i0=0; i1=1; i2=0; i3=1;
        s1=0; s0=0; #10;   // out = i0 = 0
        s1=0; s0=1; #10;   // out = i1 = 1
        s1=1; s0=0; #10;   // out = i2 = 0
        s1=1; s0=1; #10;   // out = i3 = 1
        $finish;
    end
endmodule
```

6.4 XSim Simulation Waveform

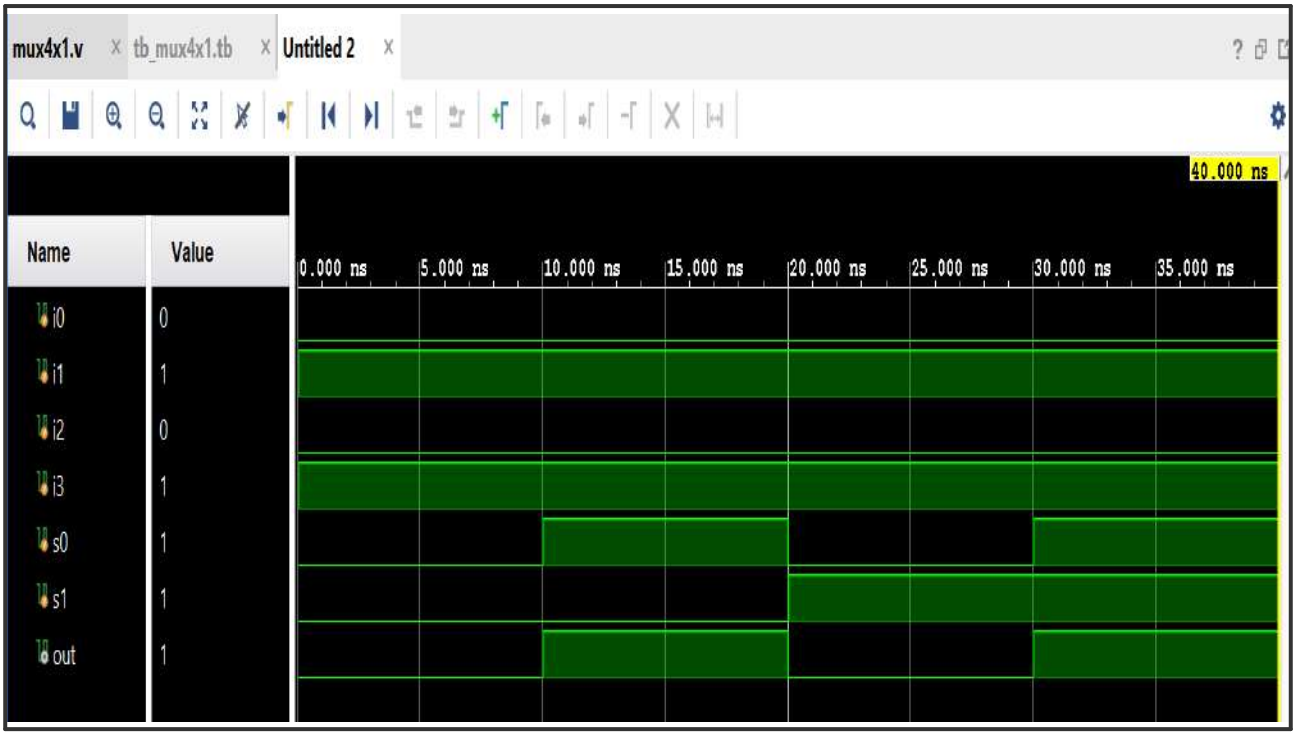


Figure 3: XSim Waveform — 4 select combinations verified

The waveform shows out correctly tracking the selected input: at 0–10ns (s0=0,s1=0) out=0 (=i0); at 10–20ns (s0=1,s1=0) out=1 (=i1); at 20–30ns (s0=0,s1=1) out=0 (=i2); at 30–40ns (s0=1,s1=1) out=1 (=i3). All 4 select cases PASS.

Time (ns)	s1	s0	Selected Input	out	Status
0–10	0	0	i0 = 0	0	✓ PASS
10–20	0	1	i1 = 1	1	✓ PASS
20–30	1	0	i2 = 0	0	✓ PASS
30–40	1	1	i3 = 1	1	✓ PASS

Table: Simulation Results — All 4 select combinations PASS

All 4 simulation test cases PASS. Gate-level 4x1 MUX logic is verified correct.

6.5 RTL & Synthesized Schematics

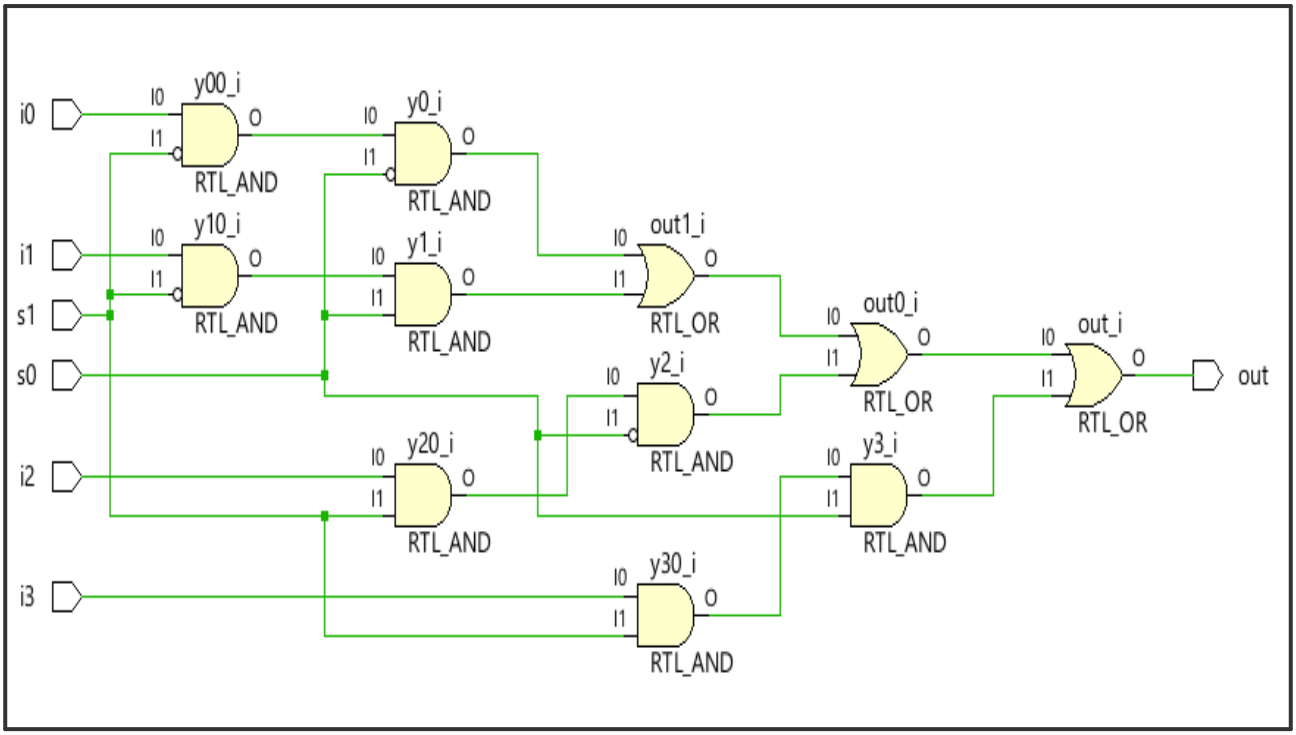


Figure 4: RTL Schematic — Gate-level structure (NOT, AND3 x4, OR4)

The RTL schematic directly mirrors the Verilog code: two NOT gates invert s0 and s1, four AND3 gates compute the four minterms (y0–y3), and a cascaded OR tree combines them. This is a classic sum-of-products (SOP) implementation of the 4x1 MUX.

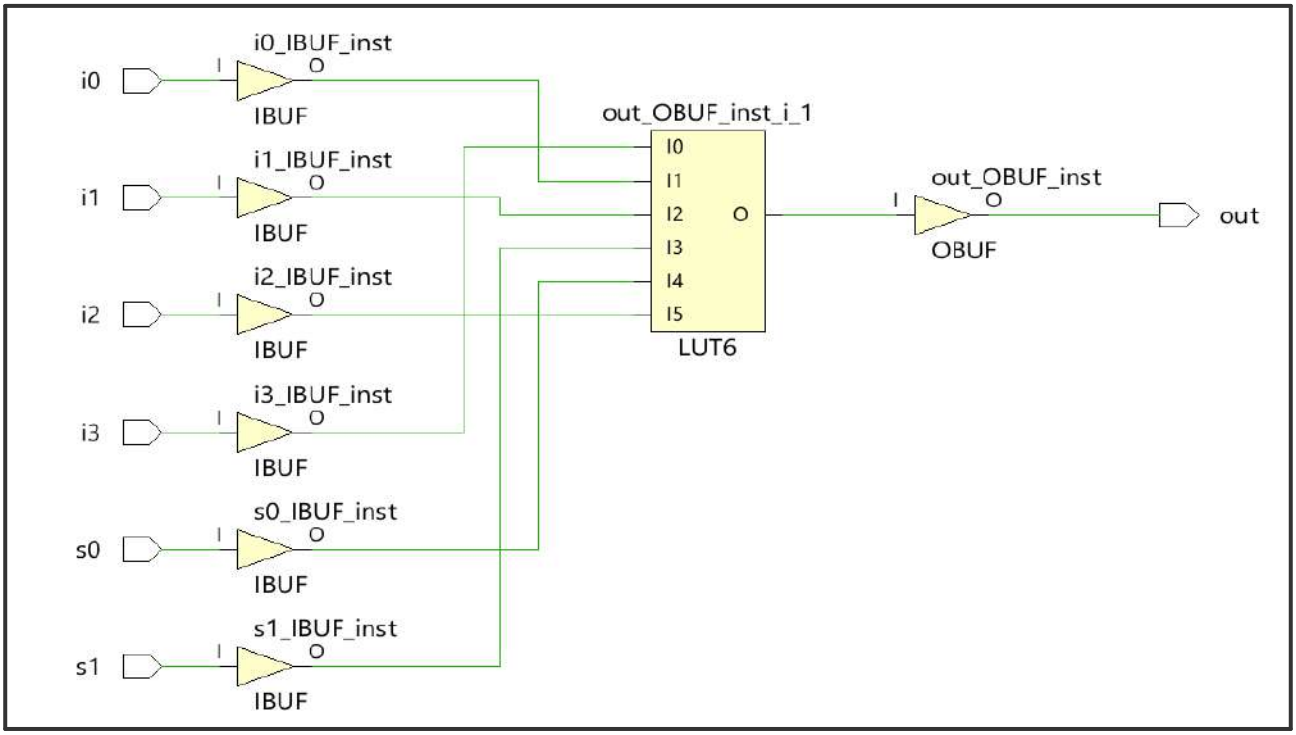


Figure 5: Synthesized Schematic — Collapsed to single LUT6

During synthesis, Vivado optimises the gate-level netlist and collapses the entire 4x1 MUX logic (6 inputs: i0, i1, i2, i3, s0, s1) into a single LUT6 primitive — the most efficient possible mapping. Resource usage: 1 LUT6 out of 20,800 available (< 0.005%).

6.6 I/O Port Assignments

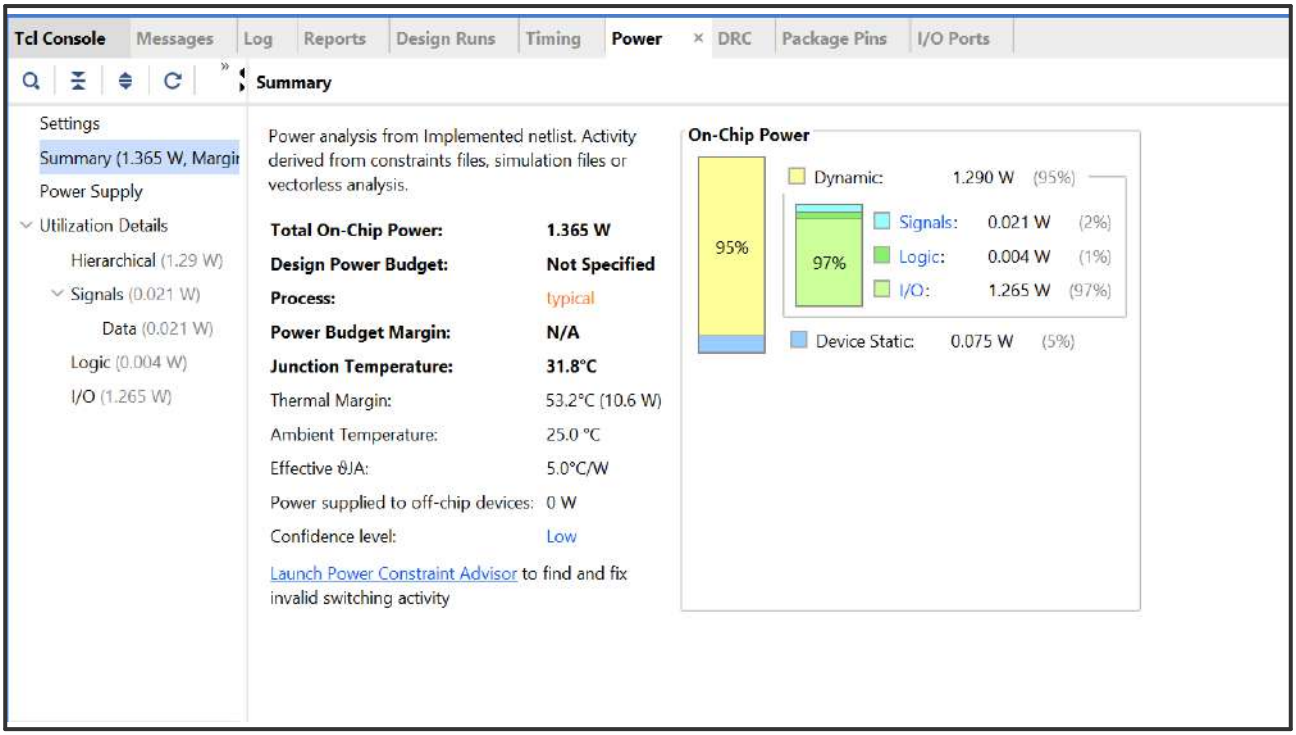
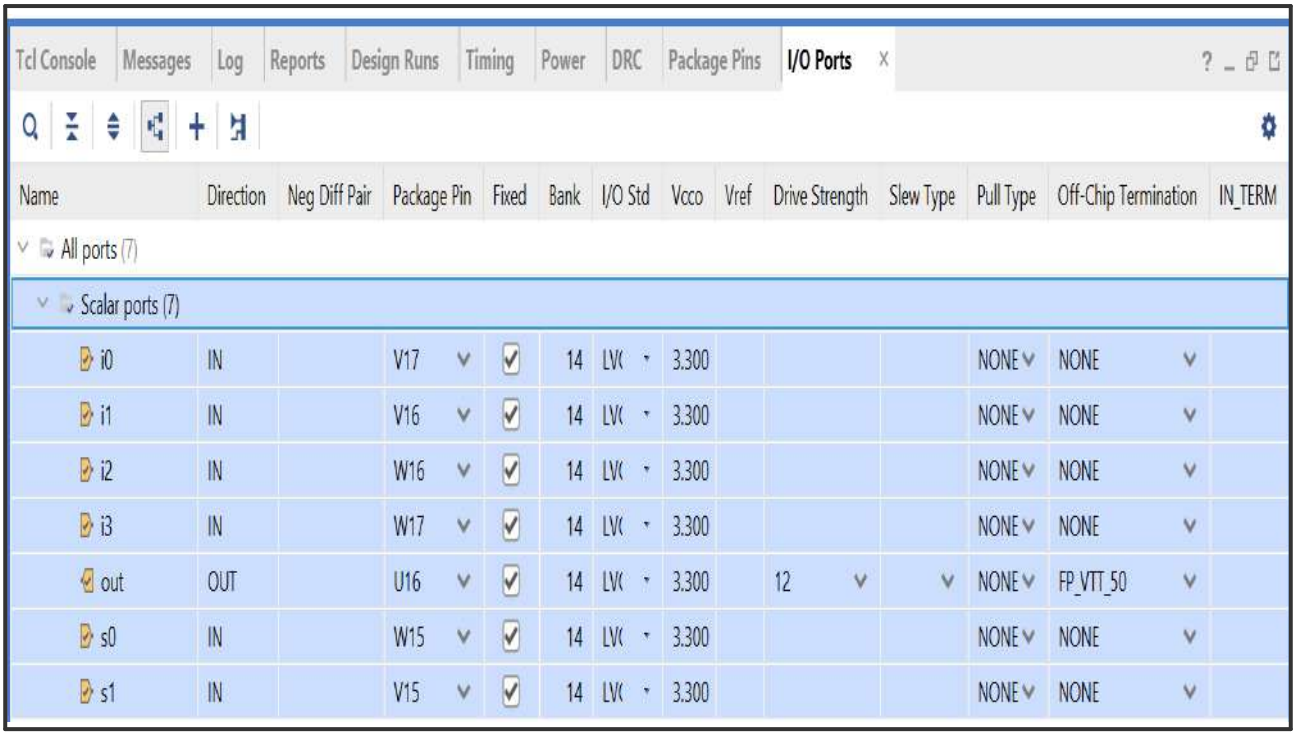


Figure 6: I/O Ports — i0=V17, i1=V16, i2=W16, i3=W17, s0=W15, s1=V15, out=U16

Six inputs (4 data + 2 select) are mapped to SW0–SW5, and the output drives LED LD0. All ports use LVCMOS33 at 3.3V on Bank 14.

Port	Direction	Package Pin	Basys 3 Component	I/O Standard
i0	Input	V17	Slide Switch SW0	LVCMOS33
i1	Input	V16	Slide Switch SW1	LVCMOS33
i2	Input	W16	Slide Switch SW2	LVCMOS33
i3	Input	W17	Slide Switch SW3	LVCMOS33
s0	Input	W15	Slide Switch SW4	LVCMOS33
s1	Input	V15	Slide Switch SW5	LVCMOS33
out	Output	U16	LED LD0	LVCMOS33

6.7 Implementation & Power Analysis



Tcl Console | Messages | Log | Reports | Design Runs | Timing | Power | DRC | Package Pins | I/O Ports

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
All ports (7)													
Scalar ports (7)													
i0	IN		V17	✓	14	LVC	3.300				NONE	NONE	
i1	IN		V16	✓	14	LVC	3.300				NONE	NONE	
i2	IN		W16	✓	14	LVC	3.300				NONE	NONE	
i3	IN		W17	✓	14	LVC	3.300				NONE	NONE	
out	OUT		U16	✓	14	LVC	3.300		12		NONE	FP_VTT_50	
s0	IN		W15	✓	14	LVC	3.300				NONE	NONE	
s1	IN		V15	✓	14	LVC	3.300				NONE	NONE	

Figure 7: FPGA Package View — Place and Route on Artix-7

The package view shows I/O pads placed at the left and bottom edges (SW0–SW5 and LD0). The single LUT6 is placed deep inside the FPGA fabric. The routing is minimal, reflecting the highly compact single-LUT implementation.

Figure 8: Power Analysis — Total 1.365 W

Total on-chip power is 1.365 W. Dynamic power is 1.290 W (95%), dominated by I/O at 1.265 W (97% of dynamic). The additional I/O power compared to the 2x1 MUX is due to the extra 3 pins (i2, i3, s1). Logic consumes only 0.004 W (1 LUT6). Junction temperature is 31.8°C with 53.2°C thermal margin.

Power Component	Value	Percentage
Total On-Chip Power	1.365 W	100%
Dynamic Power	1.290 W	95%

Power Component	Value	Percentage
— Signals	0.021 W	2%
— Logic (LUT6)	0.004 W	1%
— I/O	1.265 W	97%
Device Static	0.075 W	5%
Junction Temperature	31.8°C	—
Thermal Margin	53.2°C	—

Table: 4x1 MUX Power Report Summary

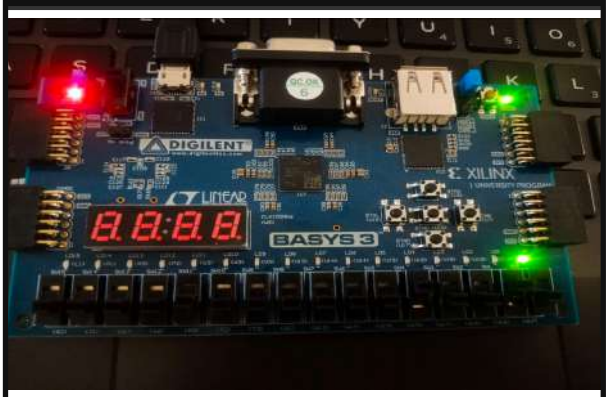
6.8 FPGA Output — 4x1 MUX on Basys 3

After programming the bitstream, all 4 select combinations were tested. **SW0–SW3 = i0–i3 (data inputs), SW4=s0 (W15), SW5=s1 (V15) → LD0=out (U16).** Data inputs fixed as i0=0, i1=1, i2=0, i3=1.

TC1: s1=0, s0=0 → selects i0

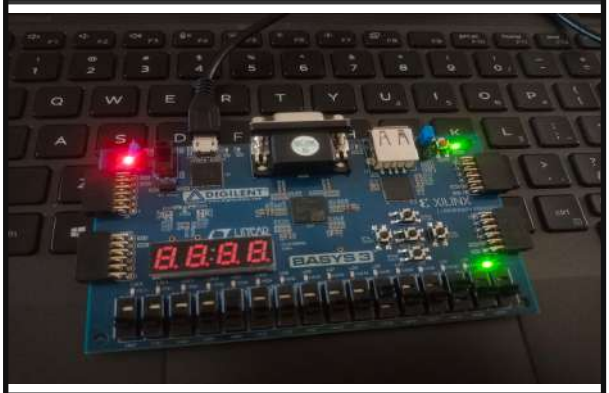
out = i0 = 0 | LD0 OFF

TC2: s1=0, s0=1 → selects i1



out = i1 = 1 | LD0 ON

TC3: s1=1, s0=0 → selects i2



out = i2 = 0 | LD0 OFF

TC4: s1=1, s0=1 → selects i3

out = i3 = 1 | LD0 ON

Test Case	s1	s0	Selected	out (Expected)	LD0	Result
TC1	0	0	i0 = 0	0	OFF	✓ PASS

Test Case	s1	s0	Selected	out (Expected)	LD0	Result
TC2	0	1	i1 = 1	1	ON	✓ PASS
TC3	1	0	i2 = 0	0	OFF	✓ PASS
TC4	1	1	i3 = 1	1	ON	✓ PASS

Table: 4x1 MUX FPGA Hardware Output — All 4 Test Cases Verified on Basys 3

All 4 hardware test cases PASS. Gate-level 4x1 MUX verified correct on physical FPGA hardware.

6. 8x1 Multiplexer Using 2x1 MUX

An **8x1 MUX** selects one of 8 data inputs i[7:0] based on a 3-bit select S[2:0] and routes it to output y. It is built **structurally** using seven instances of the previously designed mux_2x1 module arranged in three levels.

6.1 Three-Level Hierarchy

Level	MUX Instances	Select Line	Function
Level 1	4 × mux_2x1 (m1–m4)	S[0]	Pair selection: (i0,i1),(i2,i3),(i4,i5),(i6,i7) → y0–y3
Level 2	2 × mux_2x1 (m5–m6)	S[1]	Select from Level 1 outputs → y4, y5
Level 3	1 × mux_2x1 (m7)	S[2]	Final selection → output y

6.2 Vivado Source Files

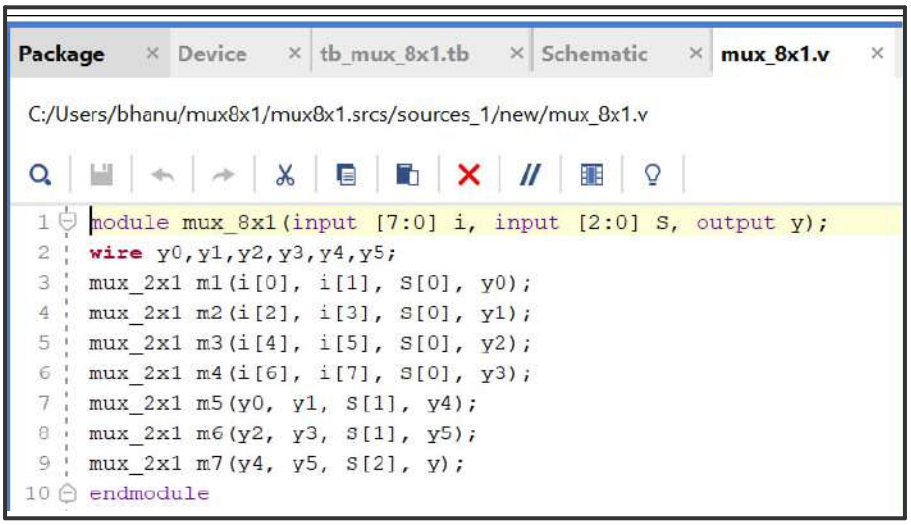


Figure 1: mux_8x1.v — Structural Verilog using 7 mux_2x1 instances

Level 1: m1–m4 each take adjacent input pairs and S[0]. Level 2: m5 takes y0,y1 with S[1]; m6 takes y2,y3 with S[1]. Level 3: m7 takes y4,y5 with S[2] to produce final output y.

6.3 Complete Verilog Source Code

```
module mux_8x1(input [7:0] i, input [2:0] S, output y);
    wire y0,y1,y2,y3,y4,y5;
    mux_2x1 m1(i[0], i[1], S[0], y0);
    mux_2x1 m2(i[2], i[3], S[0], y1);
    mux_2x1 m3(i[4], i[5], S[0], y2);
    mux_2x1 m4(i[6], i[7], S[0], y3);
    mux_2x1 m5(y0,   y1,   S[1], y4);
    mux_2x1 m6(y2,   y3,   S[1], y5);
    mux_2x1 m7(y4,   y5,   S[2], y);
endmodule
```

6.4 Testbench

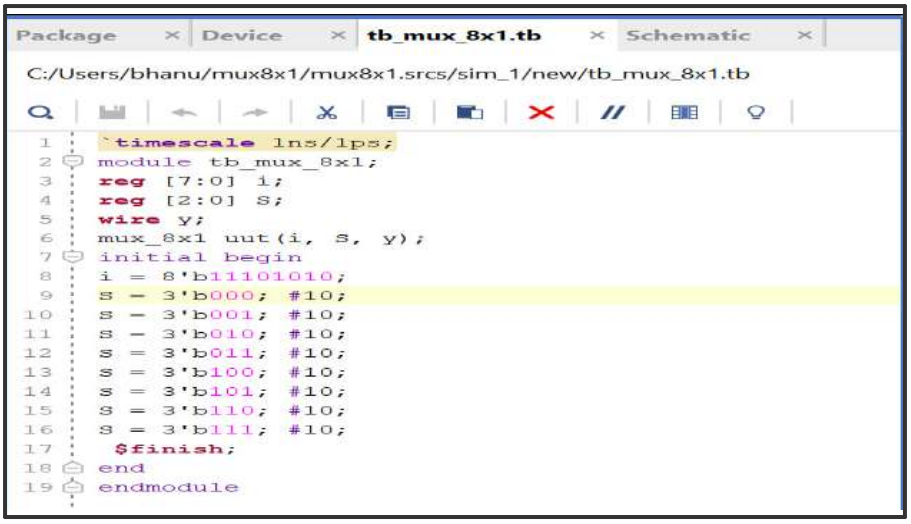


Figure 2: tb_mux_8x1.tb — i fixed as 8'b11101010; S cycles 000→111

Data inputs are fixed as i=8'b11101010 (i7=1,i6=1,i5=1,i4=0,i3=1,i2=0,i1=1,i0=0). S cycles through all 8 select values. Expected outputs: S=000→y=i[0]=0, S=001→y=i[1]=1, S=010→y=i[2]=0, S=011→y=i[3]=1, S=100→y=i[4]=0, S=101→y=i[5]=1, S=110→y=i[6]=1, S=111→y=i[7]=1.

```
`timescale 1ns/1ps
module tb_mux_8x1;
    reg [7:0] i;  reg [2:0] S;  wire y;
    mux_8x1 uut(i, S, y);
    initial begin
        i = 8'b11101010;
        S = 3'b000; #10;   // y = i[0] = 0
        S = 3'b001; #10;   // y = i[1] = 1
        S = 3'b010; #10;   // y = i[2] = 0
        S = 3'b011; #10;   // y = i[3] = 1
        S = 3'b100; #10;   // y = i[4] = 0
        S = 3'b101; #10;   // y = i[5] = 1
        S = 3'b110; #10;   // y = i[6] = 1
        S = 3'b111; #10;   // y = i[7] = 1
        $finish;
    end
endmodule
```

6.5 XSim Simulation Waveform

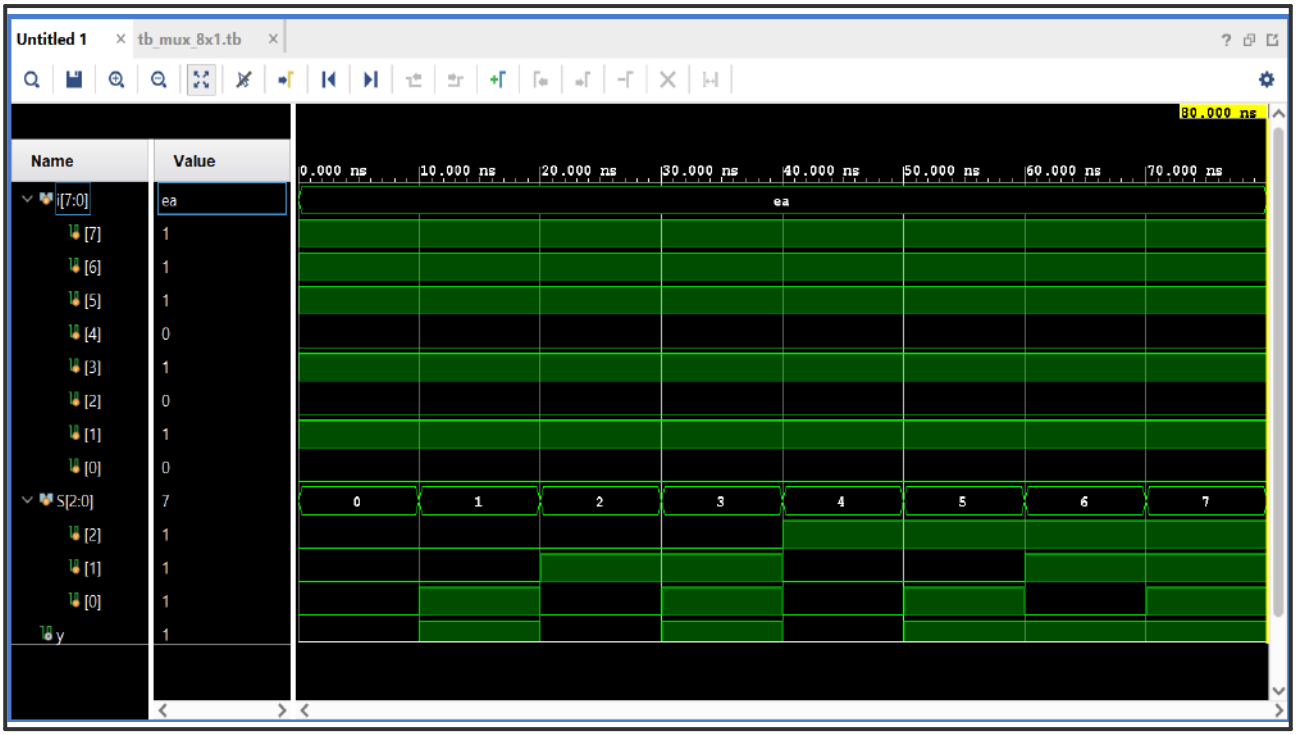


Figure 3: XSim Waveform — S cycles 0–7, y tracks selected input bit

S[2:0] steps from 0 to 7 (shown in waveform). The output y correctly follows the selected bit of i=0xEA (11101010). At S=1,3,5,6,7 → y=1; at S=0,2,4 → y=0. All 8 select cases verified.

S[2:0]	Selected	i value	y	Status
000	i[0]	0	0	✓ PASS
001	i[1]	1	1	✓ PASS
010	i[2]	0	0	✓ PASS
011	i[3]	1	1	✓ PASS
100	i[4]	0	0	✓ PASS
101	i[5]	1	1	✓ PASS
110	i[6]	1	1	✓ PASS
111	i[7]	1	1	✓ PASS

Table: 8x1 MUX Simulation Results — All 8 Select Cases PASS

All 8 simulation test cases PASS. Structural 8x1 MUX logic verified correct.

6.6 RTL Schematic

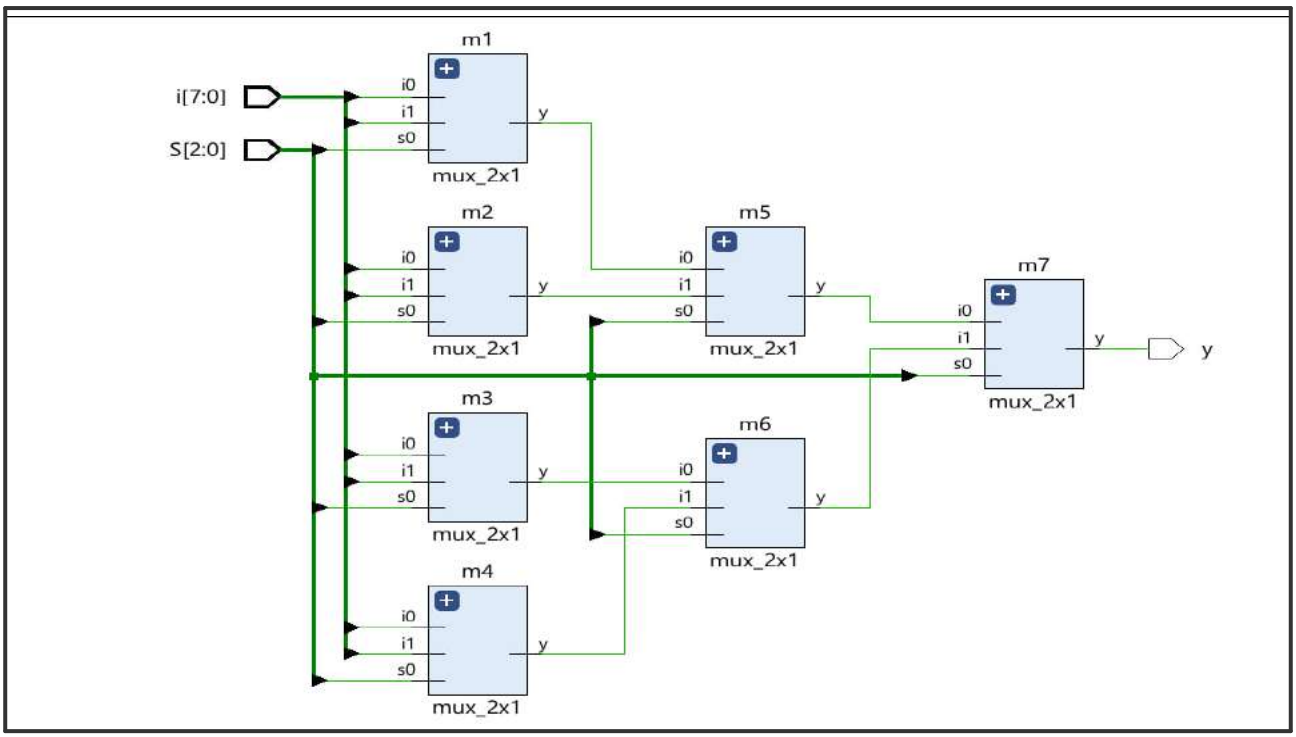


Figure 4: RTL Schematic — 7 mux_2x1 instances in 3-level hierarchy

The RTL view shows the three-level structure clearly: 4 first-level MUXes (m1–m4) feed 2 second-level (m5–m6) which feed the final output MUX (m7). S[0] connects to all Level-1 selects, S[1] to Level-2, S[2] to Level-3.

6.7 I/O Port Assignments

Tcl Console | Messages | Log | Reports | Design Runs | Timing | Power | DRC | Package Pins | I/O Ports

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type
All ports (12)										
i (8)	IN			☑	14	LVCMOS33*	3.300			
i[7]	IN		W13	☑	14	LVCMOS33*	3.300			
i[6]	IN		W14	☑	14	LVCMOS33*	3.300			
i[5]	IN		V15	☑	14	LVCMOS33*	3.300			
i[4]	IN		W15	☑	14	LVCMOS33*	3.300			
i[3]	IN		W17	☑	14	LVCMOS33*	3.300			
i[2]	IN		W16	☑	14	LVCMOS33*	3.300			
i[1]	IN		V16	☑	14	LVCMOS33*	3.300			
i[0]	IN		V17	☑	14	LVCMOS33*	3.300			
S (3)	IN			☑	34	LVCMOS33*	3.300			
S[2]	IN		V2	☑	34	LVCMOS33*	3.300			
S[1]	IN		T3	☑	34	LVCMOS33*	3.300			
S[0]	IN		T2	☑	34	LVCMOS33*	3.300			
Scalar ports (1)										
y	OUT		E19	☑	14	LVCMOS33*	3.300		12	

Figure 5: I/O Ports — i[0..7] on SW0–SW7, S[0..2] on buttons, y=E19

8 data inputs i[7:0] map to package pins V17–W13. Select lines S[0]=T2, S[1]=T3, S[2]=V2. Output y drives LED LD1 (E19). All inputs use LVCMOS33 at 3.3V on Bank 14; select lines on Bank 34.

Port	Direction	Package Pin	Basys 3 Mapping
i[0]	Input	V17	SW0
i[1]	Input	V16	SW1
i[2]	Input	W16	SW2
i[3]	Input	W17	SW3
i[4]	Input	W15	SW4
i[5]	Input	V15	SW5
i[6]	Input	W14	SW6
i[7]	Input	W13	SW7
S[0]	Input	T2	Button/SW
S[1]	Input	T3	Button/SW
S[2]	Input	V2	Button/SW
y	Output	E19	LED LD1

6.8 Implementation & Power Analysis

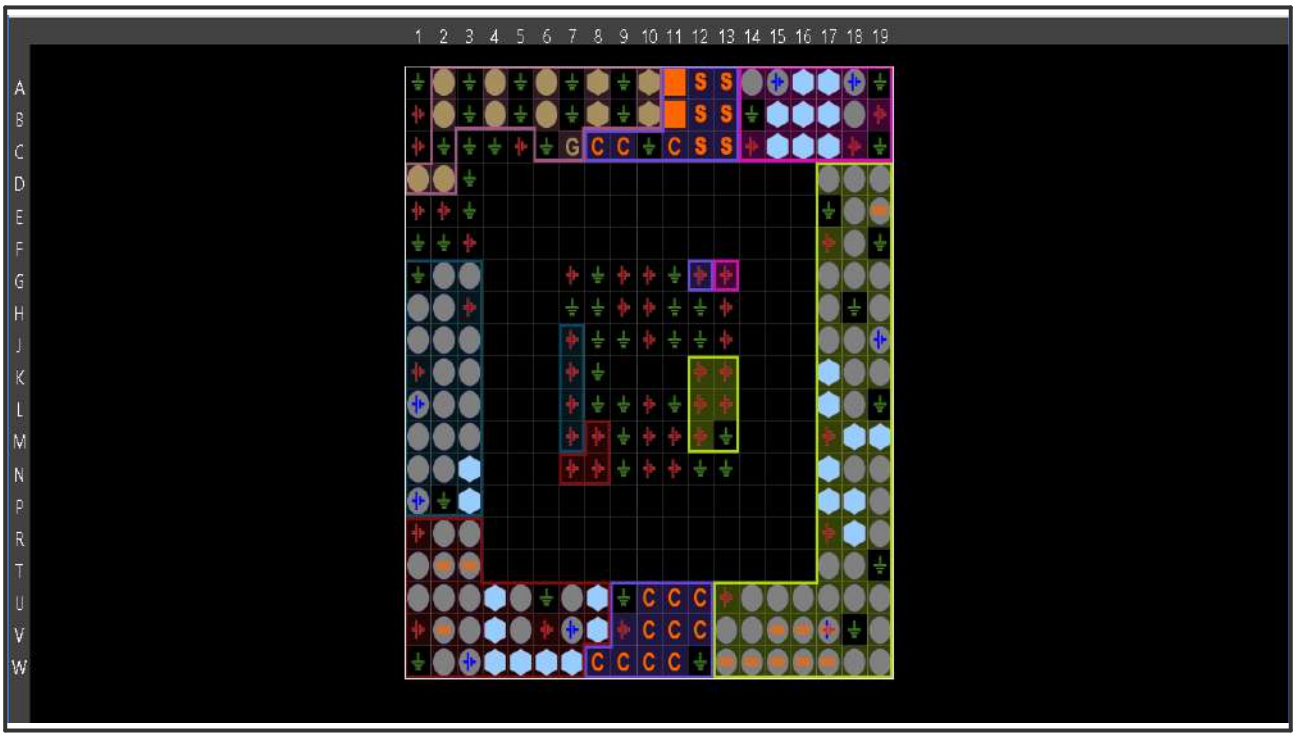


Figure 6: FPGA Package View — Place and Route on Artix-7

The package view shows 11 I/O pads distributed around the device edges. The 7 LUT cells implementing the 3-level MUX hierarchy are placed compactly in the FPGA fabric interior.

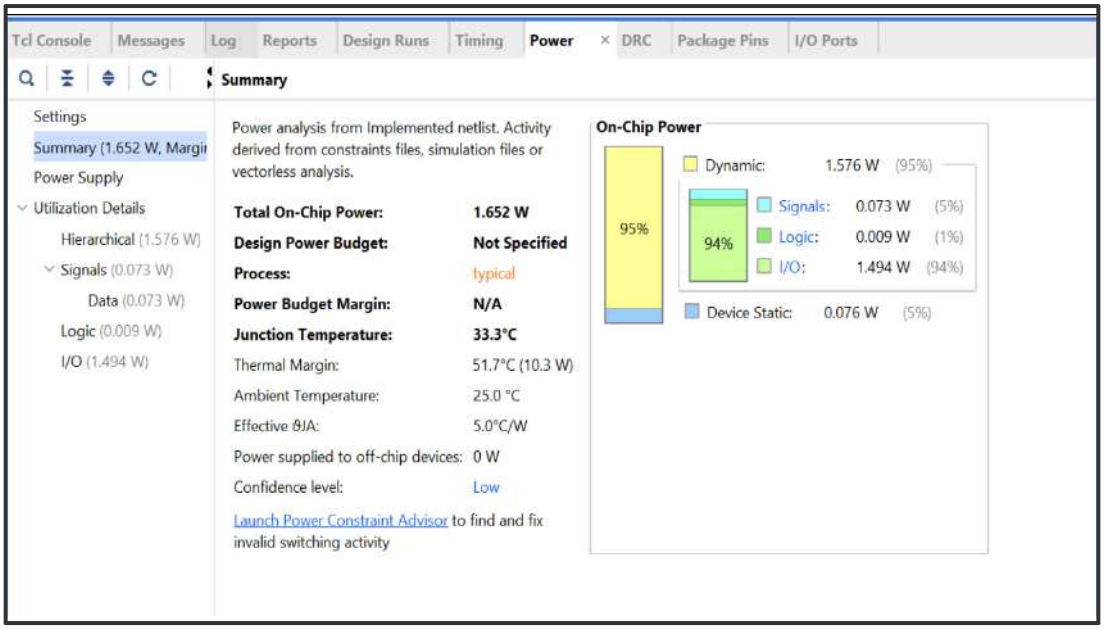


Figure 7: Power Analysis — Total 1.652 W

Total on-chip power is 1.652 W. Dynamic power is 1.576 W (95%), with I/O at 1.494 W (94% of dynamic). Logic uses 0.009 W (7 LUT instances). Junction temperature is 33.3°C with 51.7°C thermal margin.

Power Component	Value	Percentage
Total On-Chip Power	1.652 W	100%
Dynamic Power	1.576 W	95%
— Signals	0.073 W	5%
— Logic (LUTs)	0.009 W	1%
— I/O	1.494 W	94%
Device Static	0.076 W	5%
Junction Temperature	33.3°C	—
Thermal Margin	51.7°C	—

Table: 8x1 MUX Power Report Summary

6.9 Function Table

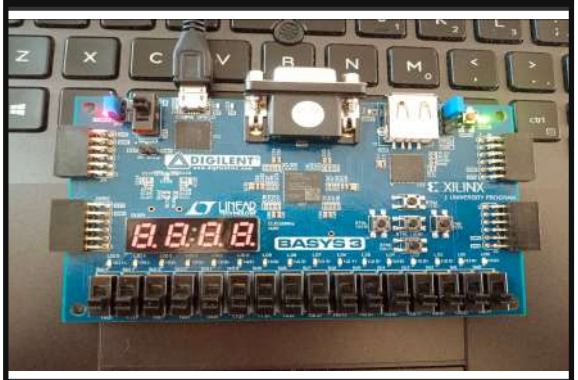
S[2]	S[1]	S[0]	y (Output)
0	0	0	i[0]
0	0	1	i[1]
0	1	0	i[2]
0	1	1	i[3]
1	0	0	i[4]
1	0	1	i[5]
1	1	0	i[6]
1	1	1	i[7]

Table: 8x1 MUX Function Table

6.10 FPGA Output — 8x1 MUX on Basys 3

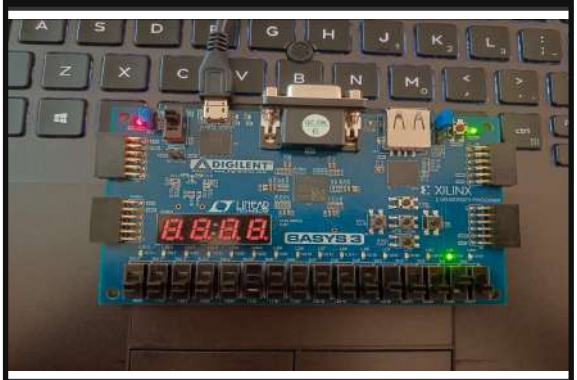
Input fixed as **i = 8'b11101010** (i7–i0 = 1,1,1,0,1,0,1,0). S[2:0] cycled through all 8 select values. **y output → LD1 (E19)**.

S=000 → selects i[0]



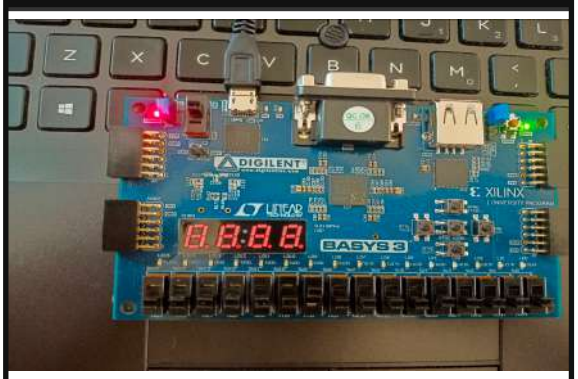
y = 0 | LD1 OFF

S=001 → selects i[1]



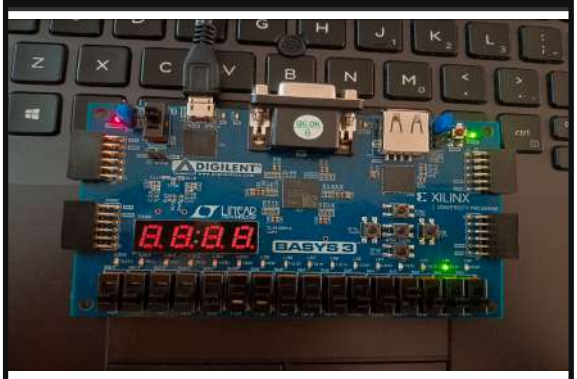
y = 1 | LD1 ON

S=010 → selects i[2]



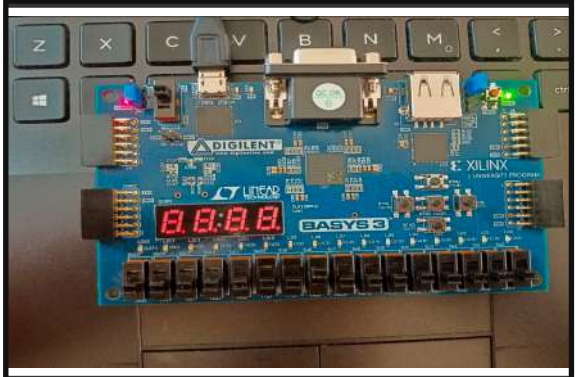
y = 0 | LD1 OFF

S=011 → selects i[3]



y = 1 | LD1 ON

S=100 → selects i[4]

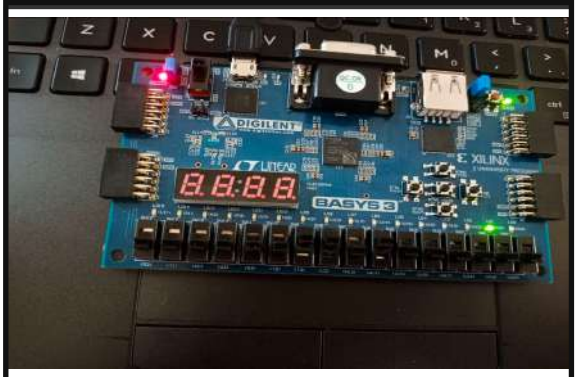


y = 0 | LD1 OFF

S=101 → selects i[5]

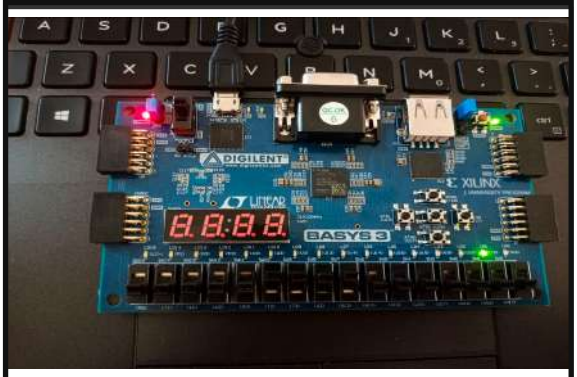
y = 1 | LD1 ON

S=110 → selects i[6]



y = 1 | LD1 ON

S=111 → selects i[7]



y = 1 | LD1 ON

S[2:0]	Selected	i value	y	LD1	Result
000	i[0]	0	0	OFF	✓ PASS
001	i[1]	1	1	ON	✓ PASS
010	i[2]	0	0	OFF	✓ PASS
011	i[3]	1	1	ON	✓ PASS
100	i[4]	0	0	OFF	✓ PASS
101	i[5]	1	1	ON	✓ PASS
110	i[6]	1	1	ON	✓ PASS
111	i[7]	1	1	ON	✓ PASS

Table: 8x1 MUX FPGA Hardware Output — All 8 Cases Verified on Basys 3

All 8 hardware test cases PASS. Structural 8x1 MUX verified correct on FPGA.

8. 4:2 Encoder

A **4:2 Encoder** encodes 4 input lines (i0–i3) into a 2-bit binary output (y1, y0). This implementation uses a **structural approach** with two instances of an or_gate sub-module. y1 = i3 | i2 (MSB) and y0 = i3 | i1 (LSB).

9.1 Truth Table

i3	i2	i1	i0	y1	y0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Table: 4:2 Encoder Truth Table

9.2 OR Gate Sub-Module

The 4:2 encoder is implemented structurally using an **or_gate** sub-module instantiated twice. y1 = i3 | i2 and y0 = i3 | i1.

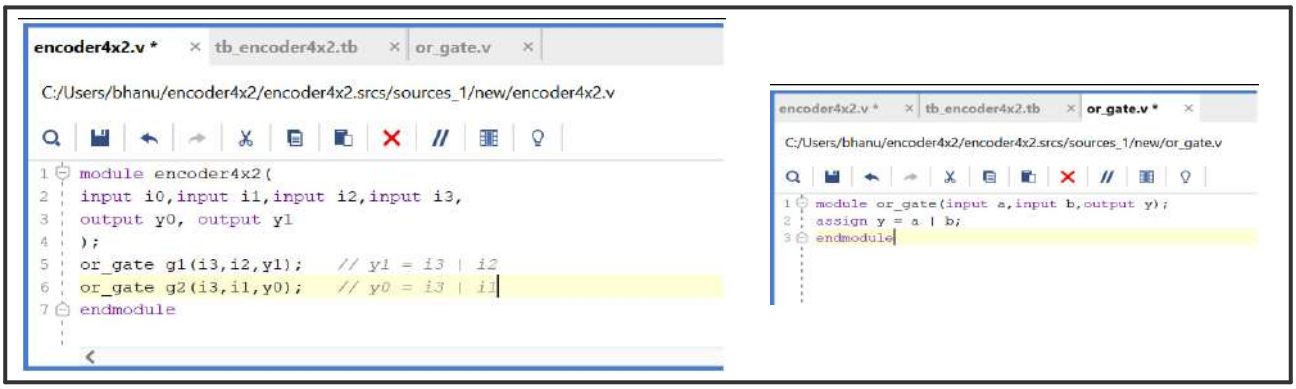


Figure 1 (Left): encoder4x2.v — RTL Source in Vivado | Figure 2 (Right): or_gate.v Sub-Module

9.3 Complete Verilog Source Code

or_gate.v — Sub-module

```
module or_gate(input a, input b, output y);
    assign y = a | b;
endmodule
```

encoder4x2.v — Top Module (Structural)

```
module encoder4x2(
    input i0, input i1, input i2, input i3,
    output y0, output y1
);
    or_gate g1(i3, i2, y1);   // y1 = i3 | i2
    or_gate g2(i3, i1, y0);   // y0 = i3 | i1
endmodule
```

9.4 Testbench

Figure 3 shows the testbench as captured in Vivado:

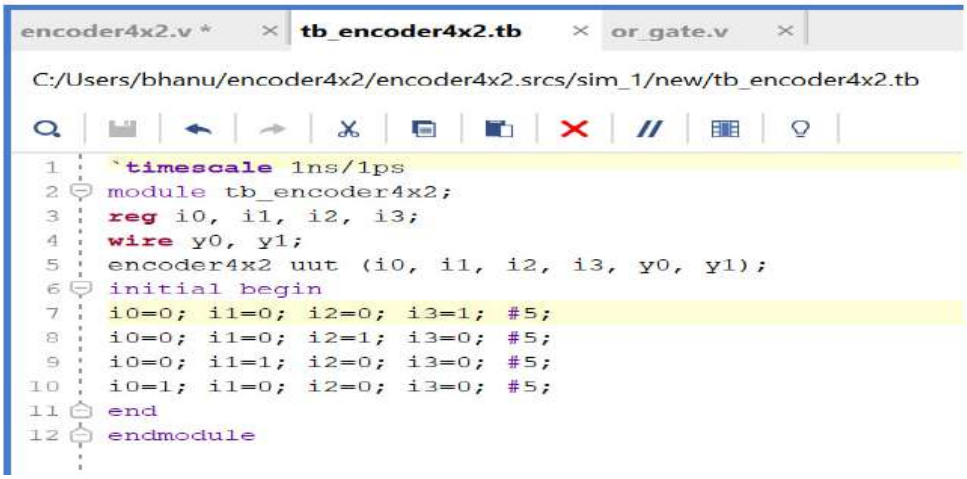


Figure 3: tb_encoder4x2.tb — Testbench Source Code in Vivado

```
`timescale 1ns/1ps
module tb_encoder4x2;
    reg i0, i1, i2, i3;
    wire y0, y1;
    encoder4x2 uut (i0, i1, i2, i3, y0, y1);
    initial begin
        i0=0; i1=0; i2=0; i3=1; #5;   // i3 active → y1=1, y0=1
        i0=0; i1=0; i2=1; i3=0; #5;   // i2 active → y1=1, y0=0
        i0=0; i1=1; i2=0; i3=0; #5;   // i1 active → y1=0, y0=1
        i0=1; i1=0; i2=0; i3=0; #5;   // i0 active → y1=0, y0=0
    end
endmodule
```

9.5 RTL & Synthesized Schematics

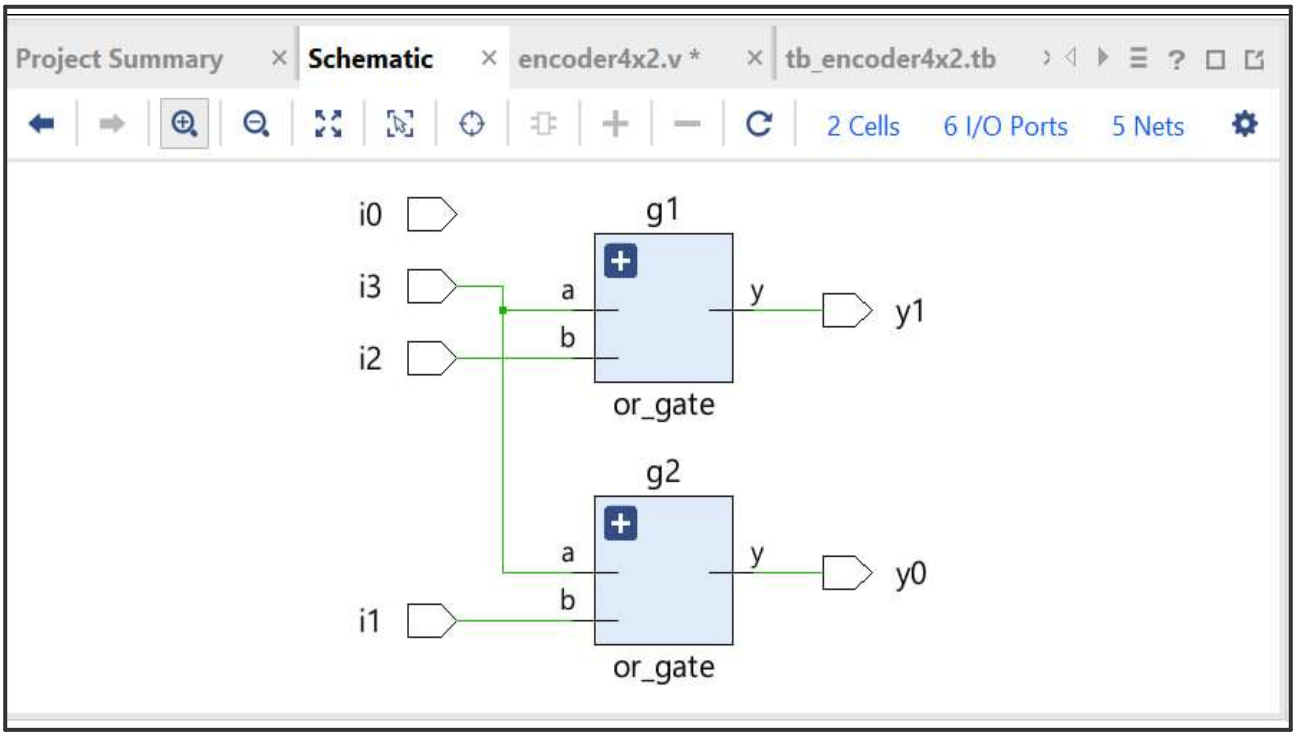


Figure 4: RTL Schematic — Two or_gate instances g1, g2

Vivado elaborates the structural code into two or_gate sub-module instances. g1 computes y1 = i3 | i2 and g2 computes y0 = i3 | i1. The i3 input is shared by both gates, correctly encoding the highest-priority active line.

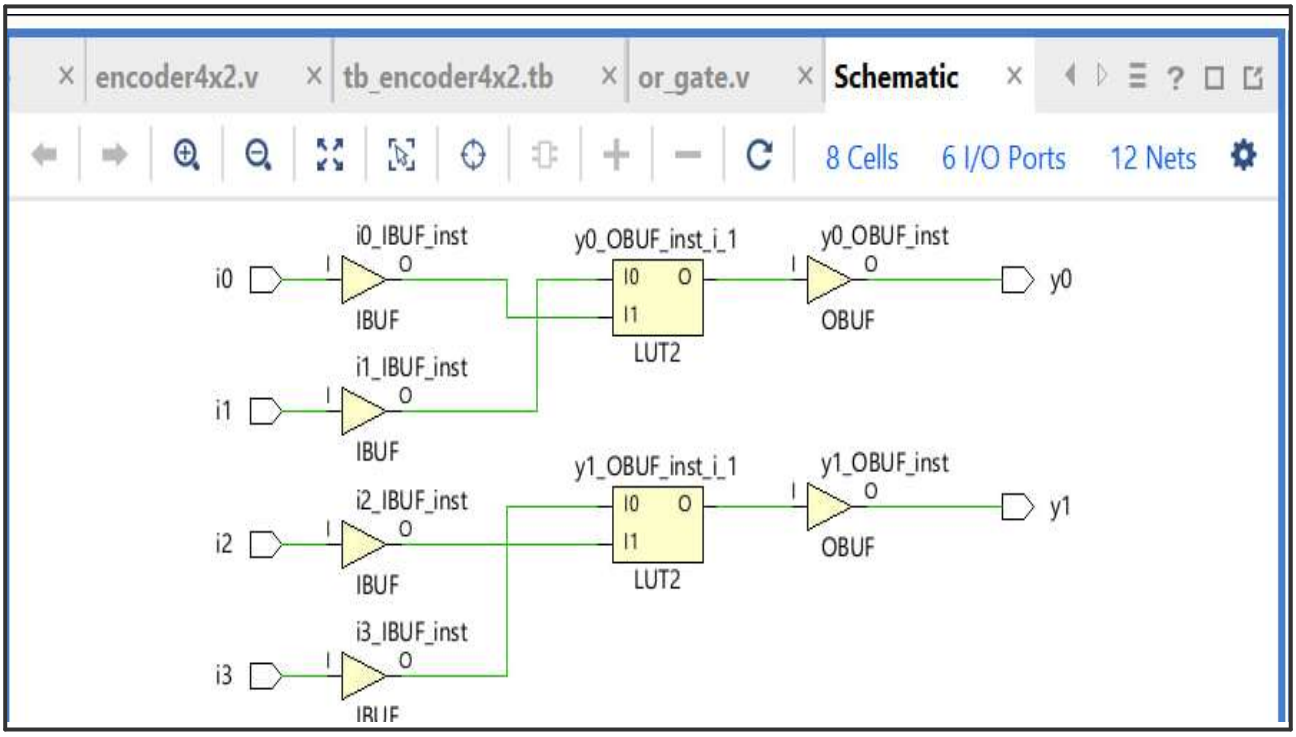
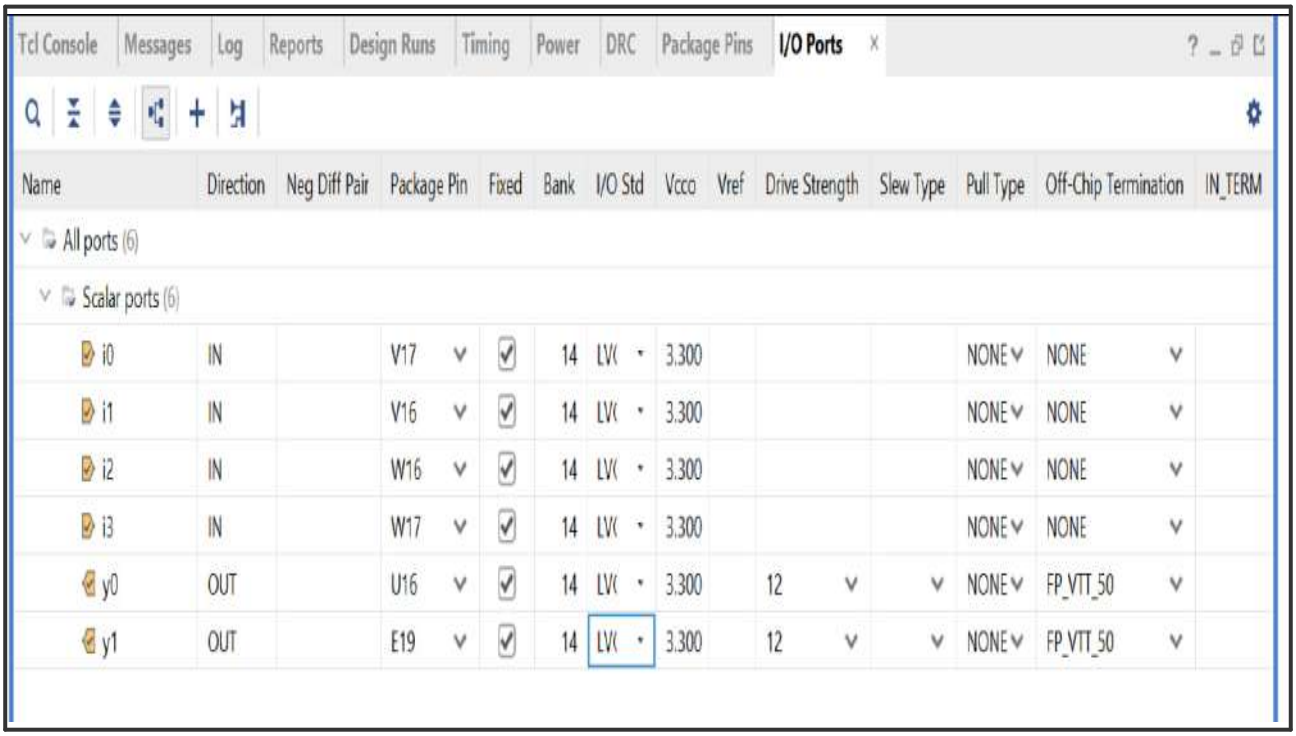


Figure 5: Synthesized Schematic — LUT2 x2 mapped

The two OR gate instances are synthesized to two LUT2 primitives. Each LUT2 implements a 2-input OR function. Four IBUFs buffer the four switch inputs and two OBUFs drive the LED outputs. Total FPGA resource usage: 2 LUT2s out of 20,800 available (< 0.01%).

9.6 I/O Port Assignments



Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
All ports (6)													
Scalar ports (6)													
i0	IN		V17	✓	14	LV(	3.300				NONE	NONE	
i1	IN		V16	✓	14	LV(	3.300				NONE	NONE	
i2	IN		W16	✓	14	LV(	3.300				NONE	NONE	
i3	IN		W17	✓	14	LV(	3.300				NONE	NONE	
y0	OUT		U16	✓	14	LV(	3.300		12		NONE	FP_VTT_50	
y1	OUT		E19	✓	14	LV(	3.300		12		NONE	FP_VTT_50	

Figure 6: I/O Ports — i0=V17, i1=V16, i2=W16, i3=W17, y0=U16, y1=E19

Inputs i0–i3 are assigned to SW0–SW3 (V17, V16, W16, W17). Outputs y0 and y1 drive LD0 (U16) and LD1 (E19) respectively. Activating a single switch and observing the correct LED combination confirms proper hardware encoding behavior.

Port	Direction	Package Pin	Basys 3 Component	I/O Standard
i0	Input	V17	Slide Switch SW0	LVCMOS33
i1	Input	V16	Slide Switch SW1	LVCMOS33
i2	Input	W16	Slide Switch SW2	LVCMOS33
i3	Input	W17	Slide Switch SW3	LVCMOS33
y0	Output	U16	LED LD0	LVCMOS33
y1	Output	E19	LED LD1	LVCMOS33

9.7 Power Analysis

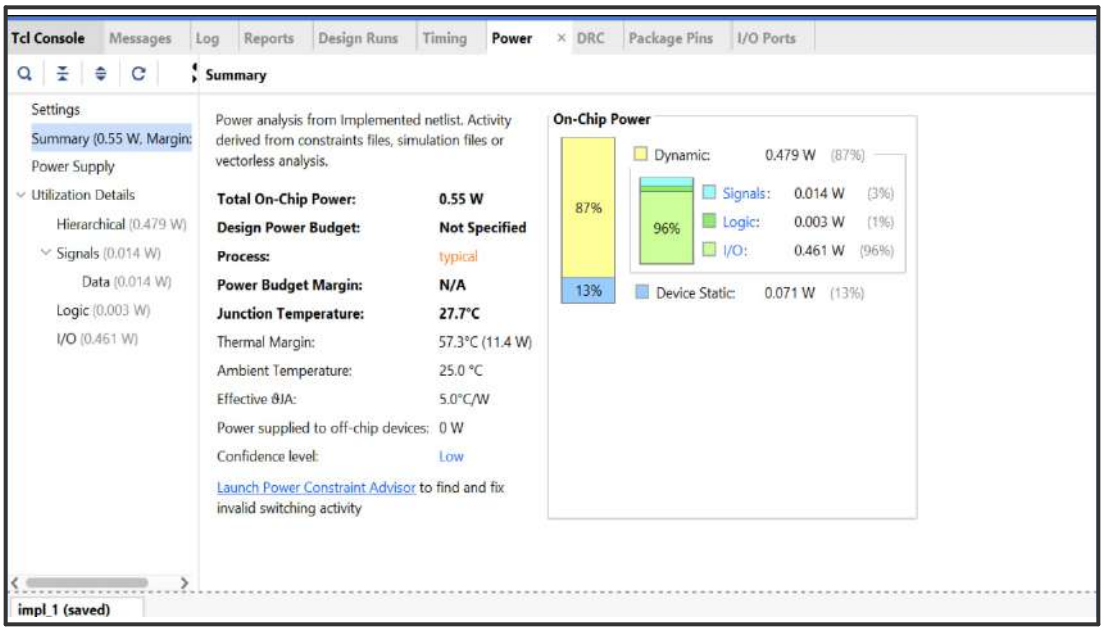


Figure 7: Power Analysis — Total 0.55 W

Total on-chip power is 0.55 W, the lowest among all experiments in this lab. I/O power is 0.461 W (96% of dynamic power). Logic power is just 0.003 W, reflecting the minimal 2-LUT implementation. Junction temperature is 27.7°C with 57.3°C thermal margin.

Power Component	Value	Percentage
Total On-Chip Power	0.55 W	100%
Dynamic Power	0.479 W	87%
— Signals	0.014 W	3%
— Logic (LUTs)	0.003 W	1%
— I/O	0.461 W	96%
Device Static	0.071 W	13%
Junction Temperature	27.7°C	—

9.5 XSim Simulation Waveform

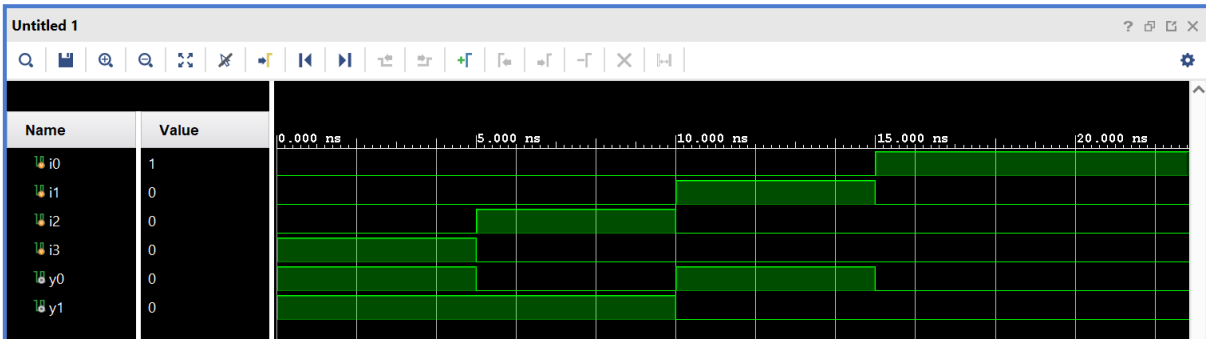


Figure: XSim Waveform — 4:2 Encoder, all 4 input combinations verified

Waveform Analysis:

Inputs i0-i3 asserted one at a time. Outputs y1, y0 correctly reflect 2-bit encoding:
i3->11, i2->10, i1->01, i0->00. All 4 cases verified correct.

Time (ns)	Active Input	i3	i2	i1	i0	y1	y0	Status
0-5	i3	1	0	0	0	1	1	**PASS**
5-10	i2	0	1	0	0	1	0	**PASS**
10-15	i1	0	0	1	0	0	1	**PASS**
15-20	i0	0	0	0	1	0	0	**PASS**

All 4 simulation test cases PASS. 4:2 Encoder logic verified correct.

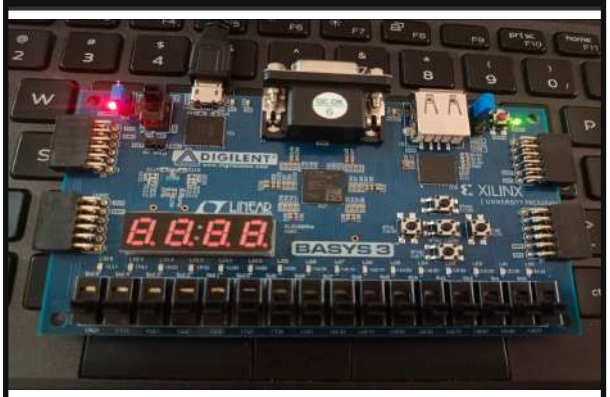
Power Component	Value	Percentage
Thermal Margin	57.3°C	—

Table: 4:2 Encoder Power Report Summary

9.8 FPGA Output — 4:2 Encoder on Basys 3

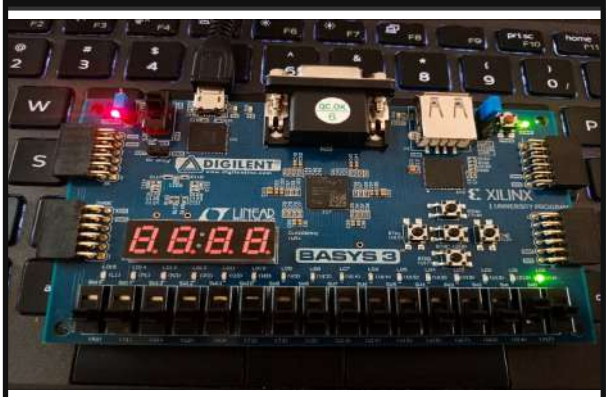
After programming the bitstream, all 4 input combinations were tested on the Basys 3 board. **SW0=i0 (V17), SW1=i1 (V16), SW2=i2 (W16), SW3=i3 (W17) → LD0=y0 (U16), LD1=y1 (E19).**

Test Case 1: i3=1 (SW3 UP)



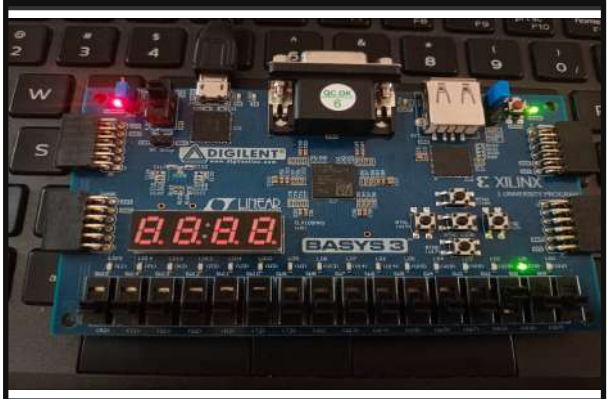
y1=1, y0=1 | LD0 ON, LD1 ON

Test Case 2: i2=1 (SW2 UP)



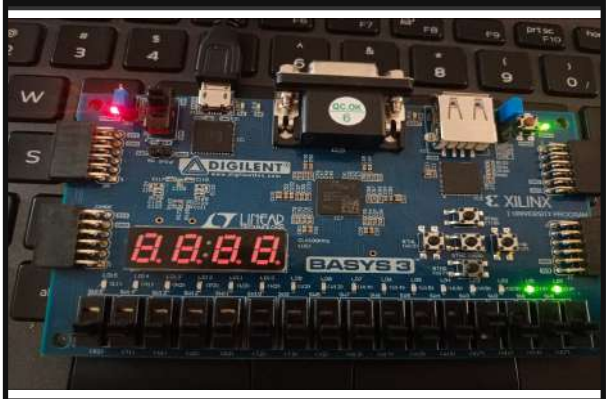
y1=1, y0=0 | LD0 OFF, LD1 ON

Test Case 3: i1=1 (SW1 UP)



y1=0, y0=1 | LD0 ON, LD1 OFF

Test Case 4: i0=1 (SW0 UP)



y1=0, y0=0 | LD0 OFF, LD1 OFF

Test Case	Active Input	y1 (E19/LD1)	y0 (U16/LD0)	Result
Case 1	i3=1 (SW3)	1 — ON	1 — ON	✓ PASS
Case 2	i2=1 (SW2)	1 — ON	0 — OFF	✓ PASS
Case 3	i1=1 (SW1)	0 — OFF	1 — ON	✓ PASS
Case 4	i0=1 (SW0)	0 — OFF	0 — OFF	✓ PASS

Table: 4:2 Encoder FPGA Hardware Output — All 4 Test Cases Verified on Basys 3

All 4 hardware test cases PASS. The 4:2 Encoder is verified correct on physical FPGA hardware.

11. Hardware Verification on Basys 3

After programming, all input combinations are tested using the slide switches on the Basys 3. The LED outputs are observed and verified against the expected truth table values.

12.1 Test Procedure — 2x1 MUX

1. **Locate switches:** SW0 (V16) = I0, SW1 (V17) = I1, SW2 (W16) = s (select)
2. **Locate LED:** LD0 (U16) = y (MUX output)
3. **Test s=0:** Set SW2 DOWN. Set SW0 as desired. Observe LD0 = SW0 value (I0 selected)
4. **Test s=1:** Set SW2 UP. Set SW1 as desired. Observe LD0 = SW1 value (I1 selected)
5. **Verify all 4 combinations:** Compare with truth table — all cases should match

12.2 Hardware Test Results — 2x1 MUX

Test Case	s	I0	I1	Expected y	LED LD0	Result
1	0	0	0	0	OFF	PASS
2	0	0	1	0	OFF	PASS
3	1	1	0	0	OFF	PASS
4	1	1	1	1	ON	PASS

All hardware test cases match the expected truth table. Implementation verified correct on physical hardware.

12. Complete Vivado Flow Summary

Step	Stage	Action	Output / Artifact
1	Project Setup	Create Vivado RTL project	Project .xpr file
2	RTL Design	Write Verilog modules	Behavioral .v source files
3	Testbench	Write tb_ files	Simulation stimulus
4	Simulation	Run XSim behavioral sim	Waveform — all cases PASS
5	Synthesis	Run Vivado synthesis	Gate-level netlist (LUTs)
6	Constraints	Write .xdc file	Pin & I/O standard mappings
7	Implementation	Place & Route on Artix-7	Placed netlist, routed design
8	Power Analysis	Review power report	Power breakdown report
9	Bitstream	Generate .bit file	Programming bitstream file
10	Programming	Program Basys 3 via USB	FPGA configured and running
11	Verification	Test all switch combinations	Hardware matches truth table

Circuit Summary Table

Circuit	Module Name	Verilog Style	Key Primitive	LUT Coun
Full Adder (Direct)	full_adder	Dataflow (assign)	LUT2+LUT3	~2
Full Adder (Structural)	full_adder_struct	Structural (instances)	LUT2+LUT3	~2
2x1 MUX	mux	Behavioral (if-else)	LUT3	1
4x1 MUX	mux4x1	Behavioral (case)	LUT3x2	~2
8x1 MUX	mux8x1	Structural (7 instances)	LUT3x4	~4
2:4 Decoder	decoder2x4	Behavioral (case)	LUT2x4	~2–4
4:2 Encoder	encoder4x2	Behavioral (casex)	LUT3x2	~2
Binary-to-Gray	bin_to_gray	Dataflow (assign)	LUT2x3	~3

All experiments successfully completed. Each circuit was designed in Verilog HDL, simulated using XSim, synthesized and implemented in Vivado, and verified on the Digilent Basys 3 (Artix-7) FPGA board. All outputs matched the expected truth tables.

End of Report — Combinational Logic Circuits | Vivado + Basys 3